

Лекция 1. Об архитектуре

Экзаменационные вопросы

1. Понятие архитектуры, профессия «Архитектор»

Архитектура — совокупность важнейших решений об организации системы: компоненты, связи (между собой и с окружением), ограничения, мотивация. Формального общепринятого определения нет; **ISO/IEC/IEEE 42010**: *fundamental concepts/properties of a system embodied in its elements, relationships, and principles of design and evolution*. Архитектура есть *всегда* (может быть плохой), *эволюционирует* с системой, *гуманитарна* (нет «правильной» — есть удачная/неудачная). **[+]** актуальная редакция — **42010:2022** (entity of interest; ADF).

Зачем. Ценен не код, работающий у разработчика, а **программный продукт** (тесты, докум., сопровождение, обработка ошибок) и **системный продукт** (интеграция). Трудоемкость $\approx 1:3:9$. Архитектура нужна, когда ПО >100 К строк. Четыре функции: (1) старт реализации (экономит код), (2) управление проектом (оценка, команда, структура организации), (3) переиспользование, (4) рассуждение о качествах до кода.

Архитектор (профстандарт 06.003): согласование требований, выбор и моделирование решения, архитектурная документация, контроль реализации/испытаний, сопровождение. Это **ключевой технический специалист** + техруководство + коммуникация (нужны технические и социальные навыки), широкий кругозор и быстрое погружение в домен (у разработчика — глубина в технологии). **Должен** писать код вместе с командой; **не должен** сидеть в «башне» и микроменеджить. Граница с программистом — на уровне паттернов GoF (что делает компонент — архитектор, как реализован — программист).

vs. «программа» (только рабочий код) vs архитектура (структура + нефункц. свойства). **[+]** современные практики: ADR, C4-модель, arc42; закон Конвея (структура = коммуникации организации).

2. Архитектурные виды

Архитектурное описание строится по частям, каждая описывает систему с одной стороны (аналогия: планы здания — этажи, электрика, водопровод).

- **Вид (view)** — описание с одной определённой стороны; состоит из *описаний* разной *формальности* (не-/полу-/формальные) и разного *уровня детализации*.
- **Точка зрения (viewpoint)** — совокупность видов, единообразно рассматривающих систему. *Пример:* диаграмма классов и ER-диаграмма — два *вида* одной *точки зрения* (статическая структура).

IEEE 1016-2009 — 12 точек зрения: Контекст, Композиция, Логическая структура, Зависимости, Информационная структура, Использование шаблонов, Интерфейсы, Структура системы, Взаимодействия, Динамика состояний, Алгоритмы, Ресурсы. **Все необязательны:** обязательны лишь общие сведения (назначение, границы, контекст) + то, что архитектор считает важным. Принцип — *не документировать лишнее* (виды стареют).

UML — общий промышленный стандарт: читать диаграммы должны все, рисовать — обязательно для архитектора. **[+]** готовые наборы видов: **4+1** (Kruchten: Logical/Process/Development/Physical + Scenarios), **C4** (Simon Brown — 4 уровня масштаба), терминология **42010:2022** (viewpoint / view / model kind / ADF).

3. Роль архитектуры в жизненном цикле; prescriptive и descriptive

Заблуждение: архитектор работает только на фазе проектирования («перебросил design document через стену» — практика «кочующих архитекторов»). **Современный взгляд:** архитектура — *непрерывная деятельность в центре* цикла (наравне с контролем версий), направляющая остальные деятельности.

Вход — требования: функциональные (*что*), нефункциональные (*как*: эффективность, масштабируемость, удобство, надёжность, безопасность, сопровождаемость), ограничения (технические, бизнес). **Сильнее всего архитектуру формируют НФТ.** Надёжность $\neq$ одно число: **MTBF** (наработка на отказ) и **MTTR** (восстановление) важны по-разному (зонд: критичен MTBF; телефон: оба не критичны), + **failsafe** (безопасное завершение при сбое — реактор, банк).

Prescriptive (предписывающая, as-designed) против **descriptive** (описывающая, as-built — воплощена в коде). Расхождение:

- **Дрейф (drift)** — *добавление* незапланированных решений (часто адекватны, но недокументированы, локальны).
- **Эрозия (erosion)** — *нарушение* заложенных принципов (багфиксы, рефакторинги); вреднее дрейфа, ведёт к «большому кому грязи». Похожа на энтропию: можно лишь замедлить.

При сильной эрозии — **восстановление архитектуры** по коду, рефакторинг, при потере документации — реинжиниринг (развитие системы при этом стоит).

Пример Hadoop: prescriptive HDFS — NameNode (метаданные) + реплицированные блоки на DataNode, клиент работает с DataNode напрямую. Descriptive (восстановлена по коду, ≈ 120 ч): 68 компонентов, **58 из 68 не**

соответствуют структуре пакетов — по папкам архитектуру не восстановить. **[+]** ограничения *можно* проверять автоматически — **fitness-функции** (ArchUnit, NetArchTest) в CI/CD ловят дрейф/эрозию; Hadoop сегодня: MapReduce — legacy, обработка на Spark, HDFS — для on-prem, тренд — облачные хранилища + Spark.

Лекция 2. Декомпозиция и ООП

Экзаменационные вопросы

1. Понятие декомпозиции. Модульность, связность, сопряжение, сложность

Сложность неустранима из ПО (иначе теряется ценность). **Существенная** (essential) — присуща задаче, убрать нельзя; **случайная** (accidental) — от способа решения, убирать нужно. Архитектор сложностью *управляет* (главный приём — **сокрытие**). Сложные системы (о которых вообще можно рассуждать): иерархичны; мало видов компонентов; выросли из простого. Подходы: **декомпозиция** (разбиение иерархии) и **абстрагирование** (рассуждение о типах).

Декомпозиция: *восходящая* (bottom-up — из кирпичиков; библиотеки, исследования, TDD), *нисходящая* (top-down — задача → подзадачи → заглушки → детализация; почти всегда). **Модуль** — единица кода под подзадачу (класс, компонент, .h/.c, пакет), есть *интерфейс* и *реализация*. Правила: чёткая декомпозиция, минимальный интерфейс, одна функциональность, без побочных эффектов, независимость от чужих *деталей*, сокрытие данных.

Размер — баланс (крупные → велика сложность каждого; мелкие → велика интеграция); U-кривая, ориентир «7±2». **Сопряжение (coupling)** — связи *между* модулями, цель — **слабое**; **связность (cohesion)** — связи *внутри* модуля, цель — **сильная**. **[+]** модульные системы (JPMS, ES-модули, Go, Rust); метрики LCOM/циклы — SonarQube, ArchUnit.

2. Понятия класса и объекта; абстракция, инкапсуляция, наследование

Объект — изолированная сущность с поведением и (возможно) состоянием. Свойства: **состояние, поведение, идентичность**; с состоянием связан **инвариант** (условия, истинные всю жизнь объекта; объект сам его хранит → нет public-полей). Объекты *шлют сообщения*: метод полиморфен, реально вызванный код определяется типом времени выполнения (двойная диспетчеризация → «Посетитель»). **Класс** — тип объекта (компиляция); объект — рантайм. Есть прототипные (бесклассовые) языки — JS до ES6, Self, Lua.

Абстракция — существенные характеристики с точки зрения наблюдателя (несколько на объект); пример — монопид и его реализации (числа+, строки-конкатенация, композиция функций). **Инкапсуляция** — разделяет интерфейс (контракты) и реализацию, защищает инварианты. **Наследование** = отношение **is-a** (сабтайпинг): потомок используется всюду вместо предка и *наследует его инварианты* (а не «получает public/protected члены»). Загадка квадрат/прямоугольник: связывать наследованием *нельзя* (инвариант ломает подстановку). Наследование — только для классификации и полиморфизма (хватает интерфейсов); для переиспользования кода — **композиция (has-a)**, она гибче (рантайм vs компиляция). **[+]** records/sealed/pattern matching (Java 25), default methods/трейты.

3. Принципы выделения объектов

Объектная модель домена (4 шага): (1) объекты и атрибуты — частые **существительные** → классы; (2) действия (ответственность) — **глаголы** → методы; (3) связи (вопросы «кто знает?», «где используется?»); (4) интерфейс каждого объекта. Результат — черновая диаграмма классов.

Источники абстракций (помимо домена):

- **Изоляция сложности** — сложные алгоритмы/структуры/подсистемы за фасадами (но фасад сам норовит распухнуть).
- **Изоляция изменений** — прятать изменчивое: бизнес-правила (часто!), зависимости от железа/ОС (редко, но критично — Y2K), форматы I/O, нестандартные возможности языка, сложные решения, сторонние компоненты.
- **Изоляция служебной функциональности** — фабрики, репозитории, диспетчеры, медиаторы, сервисы; бизнес-логика отдельно от деталей.

Не переусердствовать с «гибкостью на всякий случай». **[+]** это будущие шаблоны (Фабрика, Фасад, Адаптер); гексагональная/чистая архитектура и DDD — современное оформление «изоляции».

4. Принципы SOLID. Закон Деметры

- **S — Single Responsibility**: один класс — одна ответственность (одна причина меняться), полностью инкапсулирована; роль — одним предложением.
- **O — Open/Closed**: открыт для расширения, закрыт для изменения; стабильный интерфейс не менять, расширять через точки расширения/наследование (иначе → God object).
- **L — Liskov Substitution**: подтип заменяет базовый тип, не зная об этом; реализует контракт и инварианты предка. Пример: проверяемые исключения в Java (потомок не бросает незадекларированное).
- **I — Interface Segregation**: клиент не зависит от неиспользуемых методов; разные группы → разные интерфейсы (модем: IConnect, ISend). Не дробить механически.

- **D — Dependency Inversion:** верх и низ зависят от *абстракций*. Switch зависит не от Light, а от интерфейса Switchable; зависимость приходит снаружи (**DI**); IoC-контейнеры.

Закон Деметры — «не разговаривай с незнакомцами»: *A* не лезет к *C* через *B*. `book.pages.last.text` (плохо, «крушение поезда») → `book.lastPageText()` (идеал). Не путать с **Fluent API** (Stream, LINQ, Строитель — цепочка возвращает *тот же* объект). **[+]** SOLID — Feathers/Martin; IoC сегодня: Spring (рантайм), DI в ASP.NET Core, Dagger/Hilt и Micronaut/Quarkus (compile-time); ArchUnit проверяет DIP/циклы.

Лекция 3. Моделирование и UML

Экзаменационные вопросы

1. Моделирование. Визуальные модели, виды моделей, метафора визуализации

Модель — упрощённое подобие объекта для изучения некоторых свойств. Свойства: содержит *меньше* информации, *целенаправленна*, *субъективна*, *ограничена*. «All models are wrong, some are useful» — ценна *полезность*, не полнота. Польза в ПО — **управление сложностью** (взгляд с разных точек зрения) и анализ/тест до первой строки кода.

Виды моделей (по формальности): *естественный язык* (выразителен, но неформален и для автообработки бесполезен); *неформальная графика* (PowerPoint/Visio — проста, но неоднозначна, ловушка: кажется строгой); *формальная графика* (UML, ER, IDEF0 — стандартна, строгий синтаксис, но часто без семантики); *формальный текст* (AADL — анализ свойств, но многословен); *исполнимая* (LabVIEW — уже программа). Опасность — «архитектурный паралич».

Три понятия: метафора визуализации (договорённость, как изображать сущности — в UML класс это прямоугольник из 3 частей); **точка зрения моделирования** (какой аспект и для кого); **семантический разрыв** (модель не содержит всего для исполнения). **[+]** новый вид — diagrams-as-code (Mermaid, PlantUML, Structurizr/C4, D2); LLM генерируют диаграммы из кода/текста (тезис «NLP далеко» устарел).

2. Язык UML. Диаграммы классов

UML — стандарт де-факто, ~14 нотаций, единая *метамодель*; консорциум **OMG** (CORBA, BPMN). Расширения: **профили** (лёгкие) и **метамоделирование** (→ SysML). «Три амиго»: Буч, Рамбо (OMT → классы), Якобсон (OOSE → случаи использования); процесс RUP. Вехи: 1.1 (1997), 2.0 (2005), **2.5.1** (2017, актуальна **[+]**), UML 3.0 нет). Диаграммы: **структурные** (классов, компонентов, пакетов, объектов) и **поведенческие** (состояний, последовательностей, активностей, случаев использования).

Диаграмма классов: класс — 3 отдела (имя/поля/методы); видимость + public, - private, # protected, ~ package; кратность (1, 0..1, *, 1..*) у нужного конца. **Обобщение (наследование)** — стрелка на **ПРЕДКА**. Можно не показывать несущественное. **Атрибут** ≡ **ассоциация** (атрибут — связь не важна; ассоциация — важна). Интерфейс: блок «interface» + реализация *пунктиром* или «леденцовая» нотация. Зависимость — пунктир, уточняется стереотипами («call», «create», «realize», ...). **Агрегация** (открытый ромб, без владения) vs **композиция** (закрашенный, владение); ромб — у контейнера; чаще хватает простой ассоциации. Шаблоны (пунктирный прямоугольник, «bind»), перечисления («enumeration»). **[+]** UML сегодня чаще как код (PlantUML, Mermaid).

3. Диаграммы объектов и пакетов UML

Диаграмма объектов — *снимок* памяти в момент времени (множество всех состояний задаёт диаграмма классов); по смыслу поведенческая, но считается структурной. Объект — как класс, но **имя подчёркнуто**: имя: Тип, у атрибутов — значения. Применения: (1) проиллюстрировать классы (видно, что это «дерево»); (2) разобрать реальные объекты домена до диаграммы классов (затем классы — «наведение классификации»).

Диаграмма пакетов — структурная; **пакет** ≈ пакет Java / пространство имён C++. Показывают зависимости и содержимое. **Зависимость** (пунктир) — пакет связан с другим; стрелка **реализации** — в пакете-предке есть предок/интерфейс для класса из пакета-потомка (часто: пакет-предок содержит интерфейсы/абстрактные классы). Полезны для визуализации зависимостей между пакетами.

4. Диаграммы компонентов UML

Самые полезные для архитектора. Компонент — структурно связанное и *больше* класса (пакет, сборка, .dll, .so, веб-сервис) — из чего состоит высокоуровневая архитектура. Точного определения нет, но интуитивно понятно. Компонент имеет **предоставляемые** («леденец», кружок) и **требуемые** («гнездо», полукруг) интерфейсы; их стык — **сборочный коннектор**. Бывают вложенность, **порты** и **делегирование** (порт перенаправляет запрос вложенному компоненту; чаще рисуют только леденцовую нотацию).

Почему полезнее диаграмм классов: переживают большинство рефакторингов (класс переименовали — диаграмму классов перерисовывать; компоненты неизменны до глобальной переработки раз в несколько лет). Это вид «с высоты птичьего полёта» (пример — QReal: связи = зависимости по сборке, каждый компонент — отдельный .dll/.so). **[+]** соответствует уровню Container/Component в C4; для микросервисов компонент = сервис с провайдируемыми/требуемыми API (REST, gRPC); ведут через Structurizr/Mermaid.

Лекция 4. Моделирование и анализ предметной области

Экзаменационные вопросы

1. Диаграммы развёртывания UML

Показывают размещение логических элементов (компонентов, исполнимых файлов) на физических/виртуальных **узлах** (параллелепипеды: компьютеры, БД, среды выполнения). Внутри узлов — **артефакты** (бинарники, компоненты; подробно или кратко именем). **Связи только между узлами** — каналы связи с указанием *протокола*. Свойства — в фигурных скобках. **Польза в анализе:** железо часто известно заранее и определяет архитектуру; и **обратное проектирование** (по знаниям админов восстановить высокоуровневую архитектуру). **[+]** сегодня топология — это код; Docker, манифесты Kubernetes, Terraform/Helm; в C4 есть Deployment diagram.

2. Диаграммы случаев использования UML

Из **OOSE** (Якобсон, 1992) → в UML. Три вещи: **акторы** (внешние роли/системы, использующие нашу), **случаи использования** (цель актора, 1–3 слова), **граница системы**. **Случай ≠ функция:** «Залогиниться» — не случай (и не цель, и нефункциональное требование). Каждый случай раскрывается в **сценарии** (что и в каком порядке делает пользователь; обычно текст / user story). **Отношения:** «include» (всегда включает другой, как вызов), «extend» (может расширять в точке расширения; уточнение include), наследование (классификация акторов/случаев). **Структура сценария:** заголовок, акторы (основной), предусловия, триггеры, основной порядок событий, альтернативы/расширения, постусловия. **[+]** сегодня — user story, Gherkin (BDD); для формальных требований — SysML (в 2025 принят **SysML v2**).

3. Диаграммы IDEF0 (контекстные)

IDEFO (семейство IDEF, ВПК США, 1970-е; ≈ SADT) — структурная декомпозиция системы/ процесса в блоки, соединённые входами-выходами. Стороны блока — **ICOM:** слева **вход** (Input — что перерабатывается), сверху **управление** (Control — что определяет, как работать), снизу **механизм** (Mechanism — что позволяет работать), справа **выход** (Output). Блок раскрывается своей диаграммой с *трассируемостью* связей. **Контекстная диаграмма** — корень: *один блок* = вся система, стрелки извне; чётко задаёт границу системы на одном экране. Чаще для бизнес-/ производственных процессов, чем для ПО. **[+]** был FIPS 183; во многом вытеснен BPMN.

4. Диаграммы характеристик (feature diagrams)

Дерево требований. Характеристики **абстрактные** (декомпозировать) и **конкретные** (реализуемые). Отношения: **обязательная** (заполненный кружок) / **необязательная** (пустой); группы **или** (or, можно несколько) и **исключающее или** (xor/alternative, ровно одна). Главное применение — **линейки продуктов** (Professional/Ultimate) и автогенерация конфигурации/кода (пример BRICS — конфигурирование ПО сервисных роботов). **Минусы:** громоздки, слабо поддерживаются CASE. **[+]** истоки — FODA (Kang, 1990); инструменты pure::variants, FeatureIDE; родственное — feature flags.

5. Feature tree (дерево характеристик)

Более простая и быстрая нотация для *первоначального* анализа (brainstorm). «Голова рыбы» — система целиком; от неё «хребет», от него — основные фичи, далее — более мелкие ответвления (отсюда **fishbone diagram**). **Взаимосвязей не показывает**, но удобна на старте; потом рисуют случаи использования, диаграмму характеристик и т.д. **[+]** современные аналоги — impact mapping, user story mapping, mind-map-инструменты.

6. Диаграммы активностей UML

Внешне блок-схема, но моделируют *поведение бизнес-процесса* (или алгоритм). **Одна из двух диаграмм UML с исполнимой семантикой** (на **сетях Петри**; вторая — автоматы). **Ветвление/ объединение** = fork/join параллельных потоков (объединение *ждёт все*; можно не объединять). **Решение/слияние** (decision/merge) — как switch: условия пишутся *над стрелками* и *взаимоисключающе*. **Дорожки (swimlanes)** — разделение работ по отделам/частям системы. **Сигналы** — асинхронное взаимодействие (посылка сразу отдаёт управление; приём ждёт; таймер — отложенный сигнал себе). **[+]** для процессов часто заменяется BPMN; flowchart-в-Mermaid для diagrams-as-code.

7. Язык BPMN

BPMN (Business Process Model and Notation) — отдельный язык OMG (BPMN 1.0 — 2004, метамодель), не часть UML, близкий родственник; набор диаграмм под бизнес-процессы. **Как продвинутые активности:**

связывает *несколько* процессов настоящими стрелками, богаче ветвления/события, качественные исключения; при большом масштабе — нанять **бизнес-аналитика**. **Три диаграммы:** процессов (process — кто что делает), хореографии (choreography — кто когда с кем общается), диалогов (conversation — кто с кем *может* общаться). **[+]** лекция устарела: BPMN 2.0 (2011) выполняется *напрямую* (Camunda 8/Zeebe, Flowable, Activiti, jBPM), перевод в **BPEL** — legacy; решения — в DMN; «процесс как код» — Temporal.

8. Диаграммы «Сущность-связь» (ER)

Концептуальная модель данных домена → схема реляционной БД (ложится семантически идеально). **Нотация Чена** (1976): прямоугольник — сущность, овал — свойство, ромб — связь (с возможными свойствами); кратность (1:1, 1:N, M:N), первичный ключ. На практике — **нотация «вороньей лапки»** (crow's foot): атрибуты *внутри* сущности (компактнее), кратность символами на концах («лапка» = многие). **[+]** diagrams-as-code: Mermaid (erDiagram), dbdiagram.io/DBML, PlantUML; ORM-фреймворки генерируют схему (code-first).

9. Концептуальное моделирование, диаграммы ORM

Object-Role Modeling (ORM) — концептуальная модель/схема БД, реже ER, но прогрессивнее для анализа. **Только сущности и связи, атрибутов нет;** сущность имеет **схему идентификации** (признак в скобках). Связи часто **n-арные** (в ER — только бинарные), играют роль таблиц; сущности играют **роли** (прямоугольники на связи); над связью — *вариант прочтения* на естественном языке (проверка разумности); полосы над ролями — **уникальность** сочетаний; ограничения (например, хог). **Устойчивее ER** к изменениям (в ER атрибут, получивший атрибуты, → переделка; в ORM — просто новая связь), удобна на ранних этапах. **Минус:** крупнее ER. **Не путать:** ORM в программировании = **Object-Relational Mapping** (Hibernate/JPA, EF Core, SQLAlchemy, Prisma; impedance mismatch — в реляционной модели нет наследования и трудно M:N).

Лекция 5. Моделирование поведения

Экзаменационные вопросы

1. Диаграммы конечных автоматов UML

Упрощённые **диаграммы Харела** (1987); *вторая* диаграмма UML с исполнимой семантикой. Моделируют **ре-активные** системы (чёткие состояния, реакция на события). Состояние — скруглённый прямоугольник; переход — событие [стражник] / действие; стражники *взаимоисключающие* (недетерминизм некорректен); псевдосостояния начала (мгновенный переход) и конца. **Внутри состояния:** entry / exit (при входе / выходе), do (пока в состоянии), **внутренний переход** (по событию, без entry/exit). **Вложенные, параллельные** (пунктир делит на подавтоматы), **история** (помнит последнее вложенное). *vs активности:* там — активности (не задерживаемся, работа в активности, один метод); тут — состояния (стабильны, работа при переходе, целый объект). **Генерация кода:** (1) гигантский switch; (2) таблица состояний + интерпретатор; (3) паттерн «Состояние». **[+]** XState (v5, JS/TS), SCXML (W3C), Spring Statemachine, Boost.SML; автокодер JPL генерирует C для Curiosity.

2. Диаграммы последовательностей UML

Взаимодействие **объектов** (не классов) во времени. **Линия жизни** (пунктир — пока объект существует) с **линиями активации** (метод объекта на стеке; одновременно несколько — рекурсия). **Сообщение** (стрелка между активациями) ведёт в *начало* активации; не выходит из неактивного и не входит «в никуда». Ось X — объекты, Y — время (сверху вниз). Создание (вызов конструктора) и удаление (деструктор / «самоубийство»). **Фреймы** «loop»/«alt»/ «opt» — цикл/ветвление/опция, но ухудшают читаемость. Почти копия языка **MSC**. **Применение:** протоколы (многопоточность, телеком), план тестирования, отладка по логам. **[+]** Mermaid (sequenceDiagram), PlantUML; распределённая трассировка (OpenTelemetry, Jaeger).

3. Диаграммы коммуникации UML

То же, что последовательностей, но «с высоты птичьего полёта»: один сценарий, та же последовательность сообщений, но объекты на **2D-плоскости**, а порядок — **числовыми индексами** (1, 1.1 — первый вызов внутри 1; 1.3 — третий). До двоеточия — имя объекта, после — тип. **Плюс:** компактнее. **Минус:** порядок во времени *не очевиден*.

4. Диаграммы составных структур UML

Продвинутые диаграммы компонентов (часто для аппаратуры из стандартных блоков с интерфейсами). Отличие: внутри объёмлющего компонента — не компоненты, а **роли**. **Роль** $\approx$ объект: ролей одного *типа* может быть несколько, по-разному связанных; в работе роль играет любой компонент правильного типа; [1] — кратность. Порты, интерфейсы, связи (зовут **разъёмами**) — как на диаграмме компонентов. Если компоненты — структура времени *компиляции*, то здесь — времени *выполнения*. **[+]** соответствует internal block diagram (ibd) в SysML (в 2025 — SysML v2).

5. Диаграммы коопераций UML

Среднее между диаграммами объектов и классов: вместо классов/объектов — **роли** (на место роли подставляется реальный объект). Показывают взаимодействие ролей *в рамках одного сценария использования* (пример «Аукцион»: роли покупатель/продавец через лоты и предложения). Два стиля: пунктирный овал-кооперация с ролями, или «ближе к диаграмме классов» — класс, *исполняющий* роли. Дополняется диаграммой последовательностей (имя_объекта / имя_роли) для временного аспекта.

6. Временные диаграммы UML

Для электронщиков и систем реального времени: события с чёткими **временными ограничениями**. **Время идёт слева направо**; объекты вертикально, находятся в состояниях. **Вариант 1** — переключение как *скачок линии* (нагляднее); **вариант 2** — *пересечение линий* с именем состояния внутри (когда состояний много). Ограничения — в {фигурных скобках}. Показывают последовательность так же наглядно, как последовательности, плюс время и смену внутренних состояний. **[+]** для цифровых временных диаграмм — diagrams-as-code WaveDrom.

7. Диаграммы обзора взаимодействия и диаграммы потоков данных

Обзора взаимодействия — *совмещение* диаграмм активностей и последовательностей: узлы потока управления — это *фреймы* взаимодействий («ref»/sd). Нужны для сложного алгоритма с ветвлениями/циклами, который фреймами в одной диаграмме последовательностей показать было бы нечитаемо.

DFD (поток данных) — не входит в UML, самая древняя/популярная. **Один** вид стрелок — поток данных; **три** сущности: **процесс** (преобразует, *кружок*), **внешняя сущность** (поставляет/потребляет, *прямоугольник*),

хранилище (две горизонтальные линии). Первый набросок архитектуры или иллюстрация. **[+]** основа threat modeling (STRIDE, Microsoft Threat Modeling Tool, OWASP Threat Dragon).

8. Диаграммы IDEF0

Декомпозиция (детализация контекстной диаграммы): блоки (имя + однозначный **номер**) и стрелки. **ICOM**: слева — *материалы* (вход), справа — *результаты* (выход), сверху — *управление* (стандарты, заказы), снизу — *механизмы/ресурсы* (станки, рабочие — нужны, но не перерабатываются). Стрелки **ветвятся** и **сходятся**; каждая внешняя стрелка продолжается на диаграмме *уровнем выше* (трассируемость до контекстной), однозначная схема именования. Чаще для бизнес-процессов; в ПО заменяются диаграммами компонентов и составных структур, но аналитики передают IDEF0 архитекторам.

9. Сети Петри

Математический формализм анализа *параллельных* программ (не только нотация). Граф из **мест** (круги), **переходов** (закрашенные прямоугольники), **токенов** (точки). **Семантика**: переход *срабатывает*, если в каждом входном месте ≥ 1 токен — забирает по токenu из входных, кладёт по токenu в выходные. Формально (P, T, ϕ) , $\phi : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$; маркировка $\mu : P \rightarrow \mathbb{N}$; срабатывание $\mu'(p) = \mu(p) - \phi(p, t) + \phi(t, p)$. **Конечный автомат — частный случай** (каждый переход: одно входное и одно выходное место). **Не тьюринг-полны** $\Rightarrow$ свойства *разрешимы*: достижимость, ограниченность/**безопасность** (≤ 1 токена), **живость** L0–L4 (нет дедлоков), реверсабельность/домашнее состояние; структурные (для любой маркировки). **Анализ**: алгебраический (вектор/матрица — быстрый, слабый: решение уравнения *ещё не значит* достижимость), структурный (сифоны/ловушки), редукция, **пространство состояний** (ω для неограниченных; граф конечен, но рост экспоненциальный). **[+]** основа process mining (van der Aalst; Celonis, ProM, PM4Py); soundness BPMN проверяют через сети Петри.

Экзаменационные вопросы. Лекция 6

Структурные шаблоны проектирования

Паттерн — повторяемое решение типовой задачи проектирования в типовом контексте; имеет имя, известен, переиспользуем в разных языках. Это идея, а не рецепт и не языковая идиома.

1. Паттерн «Компоновщик» (Composite)

Представляет иерархию *часть-целое* деревом и даёт работать с одиночными и составными объектами *единообразно*. Участники: **Component** (общий интерфейс с операциями функциональности и, обычно, управления потомками), **Leaf** (лист), **Composite** (узел со ссылками на **Component**, реализует операции через потомков). Клиент знает в основном только **Component**. Добавление нового вида узла — просто наследник, клиентский код не трогаем. Применяется повсюду: GUI-деревья, DOM, разобранный XML, AST компилятора. **Главная тонкость:** операции с потомками (**add/remove**) плохо ложатся на ООП — объявить в **Component** (тогда в **Leaf** бросать исключение или *пустое поведение* — Null Object) или в **Composite** (тогда клиенту нужны приведение типа или **getComposite()**). Прочее: список потомков обычно в **Composite** (не обязан быть списком); ссылка на родителя — доп. инвариант; разделяемые листья (Null Object) экономят память; кеш требует инвалидации.

2. Паттерн «Декоратор» (Decorator)

Динамически добавляет объекту обязанности, сохраняя *тот же интерфейс*. **Decorator** наследуется от **Component** (поэтому неотличим от него) и держит *поле* типа **Component**: неинтересные методы пересылает внутрь, нужные переопределяет, может добавлять новые методы и своё состояние. Декорировать можно и конкретный объект, и другой декоратор (вкладываются друг в друга; порядок важен: рамка+прокрутка ≠ прокрутка+рамка). Гибче наследования: возможности добавляются/убираются *в рантайме*. Плата — много мелких объектов и лишние полиморфные вызовы. **Требования:** **Component** должен быть *мал* (методы пересылаются; поля наследуются, но не используются — частая ошибка новичков), в идеале интерфейс без состояния; методы должны быть виртуальными. *Отличие от Адаптера* — интерфейс совпадает; *от Стратегии* — надевается снаружи, а не вкладывается внутрь. **[+]** Java I/O (**BufferedReader**), middleware (Express/ASP.NET), **@decorator** в Python.

3. Паттерн «Стратегия» (Strategy)

Инкапсулирует *алгоритм* в объект, чтобы менять его *в рантайме* и изолировать его сложность. **Context** держит поле по интерфейсу **Strategy** и делегирует ему работу; смена = передать новый объект-стратегию. **Когда:** несколько взаимозаменяемых алгоритмов; близкие классы, отличающиеся поведением (вынести в стратегии, классы склеить); один сложный многоэтапный алгоритм со вспомогательными данными; много условных операторов (заменить полиморфизмом). **Тонкости:** проектирование интерфейса — передавать данные *параметрами* (стратегия не знает контекст, но получает лишнее) или *сам Context* (гибко, но круговая зависимость и течёт инкапсуляция); задать стратегию *по умолчанию* (иначе падение); стратегия часто разделяема ⇒ без своего состояния; в C++ — как *параметр шаблона* (нет *overhead* *vtable*). **[+]** Для одного метода — лямбда (лекция отмечает, что это делает паттерн избыточным).

4. Паттерн «Адаптер» (Adapter)

Делает *несовместимый* интерфейс **Adaptee** пригодным для клиента, ждущего **Target**. **Adapter** реализует **Target** и перенаправляет вызовы в **Adaptee**, возможно нетривиально преобразуя (одно действие **Target** = несколько методов **Adaptee**); может добавлять и своё. *Отличие от декоратора:* там интерфейс совпадает, тут *меняется*. **Два вида:** *адаптер объекта* (держит **Adaptee** полем — гибче, предпочтителен) и *адаптер класса* (наследуется от **Adaptee** + реализует **Target** — короче, но негибко; при **Target**-абстрактном классе нужно множественное наследование). Единственное осмысленное применение *private*-наследования C++ — адаптер класса. **[+]** Порты и адаптеры (гексагональная архитектура), обёртки чужих SDK.

5. Паттерн «Заместитель» (Proxy)

Подставляет вместо реального объекта *суррогат* с *тем же интерфейсом*, чтобы контролировать доступ к нему. **Proxy** умеет добраться до реального объекта или *создать* его при нужде; что может — делает сам, остальное перенаправляет. *Отличие от декоратора:* прокси ссылается на *конкретный* класс и управляет его созданием (прокси на прокси не надевают). **Виды:** удалённый (объект на другой машине), виртуальный (ленивое создание тяжёлых), защитный (контроль доступа), умный указатель (подсчёт ссылок, блокировки, *copy-on-write*). Пример: **ImageProxy** знает размер, грузит картинку лениво. **Тонкости:** реализует *весь* (часто громоздкий) интерфейс — помогают ⇒ **real.do()** и кодогенерация; проверять, что реальный объект уже в памяти. **[+]** Ленивые прокси ORM (Hibernate/JPA), динамические прокси AOP (Spring: JDK/CGLIB), клиенты gRPC/OpenAPI (*WCF из лекции — legacy*), **Future/Task**, **shared_ptr**, **sidecar** (Envoy).

6. Паттерн «Фасад» (Facade)

Даёт *простой* интерфейс к большой и сложной подсистеме, скрывая её внутренности. Может и *ограничивать* доступ к внутренним классам, становясь *границей* и контрактом подсистемы. Пример: `Compiler` с единственным `Compile()` сам собирает лексер, парсер, AST, генератор кода с параметрами по умолчанию; кому надо тоньше — лезет внутрь напрямую. **Когда:** упростить переиспользование подсистемы; отделить подсистемы (в многоуровневой архитектуре уровни общаются через фасады). *Не* брать, если в фасаде получаются десятки несвязных методов. **Тонкости:** *Абстрактный фасад* (интерфейс + фабрика) скрывает все конкретные классы и даёт замену подсистемы; сделать фасад *единственной* точкой входа сложно (`internal`-классы, пространство `details` в C++ — компилятор соглашения не проверяет). **[+]** API Gateway, BFF, SDK-клиенты.

7. Паттерн «Приспособленец» (Flyweight)

Позволяет вместить *много мелких объектов*, разделяя их: состояние делят на **внутреннее** (intrinsic — общее, в приспособленце, немутабельно) и **внешнее** (extrinsic — зависит от контекста, хранится у клиента и *передается* в операции). Клиент берёт приспособленцев только из `FlyweightFactory` (пул), сам не создаёт. Пример: буквы — из пула; шрифт/цвет хранит `run` (промежуток), он ссылается на `style`; внешнее состояние буквы вычисляется по её `run`. **Условия применимости (все сразу):** много мелких объектов; осмысленное деление внутреннее/внешнее; внешнее вычислимо/компактно; идентичность не важна; семантика типа-значения; измеримый выигрыш (паттерн сильно усложняет код). **Тонкости:** внутреннее состояние немутабельно; `equals` + `hashCode` (равные $\Rightarrow$ равные хеши). **[+]** Интернирование строк, кеш `Integer` в Java ($-128..127$): `valueOf(10)==valueOf(10)`, но `valueOf(150)!=valueOf(150)` — из-за разделения и ссылочного `==`.

Экзаменационные вопросы. Лекция 7

Порождающие шаблоны проектирования

1. Паттерн «Мост» (Bridge)

Разделяет *две независимые иерархии* — **абстракцию** (высокоуровневое, с чем работает клиент) и **реализацию** (**Implementor**, низкоуровневое, через что абстракция работает) — связывая их *композицией*. Фраза «отделение абстракции от реализации» осмысленна только со структурной диаграммой. Обе стороны — расширяемые иерархии классов. Пример: интерпретатор роботов — слева логические сенсоры, справа протоколы связи (Bluetooth/Wi-Fi/USB), мост — композиция; число классов из *произведения* становится *суммой*. **Отличие от соседей:** «Стратегия» — абстракция одна, реализации без сложной иерархии; «Заместитель» — интерфейсы совпадают, просто меняем реализацию; о «Мосте» говорят, когда расширяются *обе* стороны. **Implementor** внедряют: сама абстракция / по умолчанию + сеттер / клиент или IoC / фабрика (отвязывает полностью, скрывает платформозависимые реализации). **[+]** PImpl в C++ (вырожденный мост: указатель на реализацию в .cpp ⇒ быстрая перекомпиляция и бинарная совместимость; так устроена Qt); JDBC над драйверами, SLF4J над бэкендами.

2. Паттерн «Фабричный метод» (Factory Method)

Creator делает основную работу, но не знает точный тип **Product**: объявляет *виртуальный* `factoryMethod()`, подклассы (**ConcreteCreator**) переопределяют его, возвращая нужный **ConcreteProduct** (обычно в одну строку). Эволюция идеи: «виртуальный конструктор» с прокси-полем и if-цепочкой (плохо: нарушен SRP, лишняя косвенность, нельзя сделать абстрактным) → статический `create(id)` (приемлемо) → виртуальный `factoryMethod` (паттерн). GoF-пример: `Application.CreateDocument()`. **Когда:** **Creator** делает работу, но часть делегирует продукту; способ параметризовать объект другим, не передавая явно. **Тонкости:** **Creator** не обязан быть абстрактным (но забудут переопределить — будет сюрприз); *нельзя* звать `factoryMethod` в конструкторе (подкласс ещё не готов); альтернативы — параметр шаблона (C#/C++, не Java). **[+]** Лучший современный способ — лямбда: `Supplier<T>` / `Func<T>`.

3. Паттерн «Абстрактная фабрика» (Abstract Factory)

Создаёт *семейства* связанных продуктов. **AbstractFactory** объявляет создание абстрактных продуктов, **ConcreteFactory** (по одной на семейство) создаёт конкретные; клиент знает только абстракции (конкретных классов может не видеть — плагины). Сменить семейство = передать другую фабрику. Пример: стили оформления редактора (Windows/Linux), один switch при старте выбирает фабрику. **Когда (любое условие):** система не зависит от способа создания объектов; конфигурируется типом продуктов (стили, плагины); связанные объекты используются вместе (не перепутать стиль); библиотека раскрывает только интерфейсы. **Габри:** новый *вид* семейства — легко (новая фабрика); новый *тип* продукта — больно (метод в каждую фабрику); параметр-выбор `createWidget(type)` ломает типобезопасность. **Отличие от фабричного метода:** тот — метод в классе, который и так делает работу; абстрактная фабрика — выделенный класс *только* для создания (набор фабричных методов). **[+]** DI-контейнеры, плагины, темы, look-and-feel.

4. Паттерн «Одиночка» (Singleton)

Гарантирует *единственный* экземпляр и *глобальный* доступ: private-конструктор + public-static `instance()`, лениво создающий статическое поле. «Multiton» — реестр одиночек по ключу. **Реализация** (частый вопрос на собеседовании): наивная ленивая даёт *гонку*; eager static-инициализация безопасна, но без ленивости; `synchronized getInstance` — синхронизация на каждый вызов; правильно — *double-checked locking* + `volatile`. **Критика (антипаттерн):** по сути глобальная переменная — мутабельный запутывает поток данных, скрытый порядок инициализации; *мешает тестам* (нельзя замочить и внедрить зависимости); нарушает SRP; склонен размножаться, и тогда болезненный рефакторинг. **[+]** IoC-контейнер со `scope = singleton` решает большинство проблем; enum-одиночка (Bloch) потокобезопасен и устойчив к сериализации.

5. Паттерны «Ленивая инициализация» и «Пул объектов»

Ленивая инициализация — создание объекта *по требованию* (структурно как «Одиночка», но без цели уникальности/глобальности): обёртка со способом создания (лямбдой) и методом `instance()`, создающим объект при первом обращении. Те же проблемы многопоточности (double-checked locking). Нужна для экономии ресурсов, а кое-что (бесконечные структуры данных) без неё *невозможно*. **[+]** `Lazy<T>` (.NET), `lazy` (F#), `by lazy` (Kotlin), ленивость как основа Haskell, JIT.

Пул объектов — набор *переиспользуемых* объектов, дорогих в создании, но нужных ненадолго; класс-пул создаёт, выдаёт и переиспользует (или сам использует, как **ThreadPool**: $N \approx$ числу ядер, очередь задач). **Габри:** при сборке мусора пул часто *вредит* (выделение и так дёшево, а пул держит ссылки ⇒ нужен подсчёт ссылок и синхронизация); пул должен быть маленьким. **[+]** `ExecutorService` (Java), `Task/TP` (.NET), пулы соединений (**HikariCP**); *виртуальные потоки Java 21* делают пул потоков под блокирующий

6. Паттерн «Прототип» (Prototype)

Объявляет метод `Clone()` и предлагает порождать объекты *копированием существующих*. `Clone()` виртуальный $\Rightarrow$ создаёт копию того класса, у которого вызван (клиент типа не знает — ещё один «виртуальный конструктор»). Пример: палитра нотного редактора — по клику клонируется выбранный прототип и тащится на нотный стан; новый вид ноты — просто новый прототип, без `switch` и фабрик. **Реализация:** *глубокое vs мелкое* копирование — глубокое (весь граф) ожидаемо, но при круговых ссылках нужен обход с запоминанием и медленно; мелкое годится для немутабельных структур; хак — сериализация/десериализация (медленно, следить за `id`); часто есть реестр прототипов («Одиночка»). `Clone()` без параметров — у каждого прототипа свой набор параметров инициализации. **[+]** прототипы — основа объектов JavaScript; `with y record` (C#), `copy()` у `data`-классов (Kotlin), копирующие конструкторы.

Экзаменационные вопросы. Лекция 8

Поведенческие шаблоны проектирования

1. Паттерн «Строитель» (Builder)

Director (распорядитель) обходит исходные данные и *по частям* передаёт их **Builder**-у, который пошагово строит **Product**. **GetResult()** *не входит* в интерфейс **Builder**: разные строители делают несвязанные продукты, **Director** про продукт не знает, результат забирает клиент (он создал конкретный builder). Пример: **RTFReader** шлёт элементы конвертеру (ASCII/TeX); **StringBuilder** (наивная конкатенация квадратична, билдер делает одно выделение памяти). **Главное применение** — «продвинутый конструктор»: именованные параметры, значения по умолчанию, валидация, выбор реализации в рантайме (Guava **NetworkBuilder**). **[+]** fluent-билдеры, **@Builder** (Lombok); путаетесь в конструкторах — заведите **Builder**.

2. Паттерн «Наблюдатель» (Observer)

Subject хранит список наблюдателей, даёт подписаться/отписаться и нотифицирует их; **Observer** — простой интерфейс (обычно один метод), вся реакция в **ConcreteObserver**. Разворачивает управление: генератор события никого явно не зовёт. Пример: данные и несколько представлений (таблица, диаграммы) держатся консистентными. **Грбли**: утечки памяти (забыл отписаться); нотифицировать только *после* полного обновления состояния (ломается при наследовании); кто иницирует (субъект или публичный **notify()** после пакета правок); *push vs pull* (слать данные или дать спросить; приём **EventArgs**). **[+]** часто встроено: **event** (C#), сигналы/слоты (Qt), **listeners** (Java); реактивные потоки Rx.NET/RxJava; идентификация отправителя (**sender**).

3. Паттерн «Шаблонный метод» (Template Method)

AbstractClass задаёт *невиртуальный* **TemplateMethod()** — скелет алгоритма в терминах абстрактных *примитивных операций*; потомки переопределяют операции, но не меняют скелет (развитие фабричного метода). Пример: **OpenDocument** = **CanOpen** → **DoCreate** (фабричный метод) → **AboutToOpen** (hook) → **Open/DoRead**. **Грбли**: виртуальные операции (пустые по умолчанию, можно забыть переопределить) vs чисто виртуальные (обязательно переопределить); примитивы **protected**; шаблонный метод обычно не виртуальный; единый префикс имён (**Do/On**). **[+]** принцип Голливуда («не звоните нам, мы позвоним»); **lifecycle-хуки** (JUnit, React, сервлеты). *Vs Стратегия*: варьирует шаги наследованием, а не композицией.

4. Паттерн «Посредник» (Mediator)

Выделенный объект для *коммуникации* между «коллегами»: коллеги знают только его и шлют события только ему, он рассылает дальше. Превращает *квадратичную* зависимость связей в *линейную*, абстрагирует и стандартизует общение. Пример: окно выбора шрифта — **FontDialogDirector** синхронизирует превью, гарнитуру, начертание, размер; рассылает только заинтересованным. Абстрактный медиатор чаще не нужен. **Грбли**: соблазн сделать **God Object** — медиатор должен *только* организовывать общение, без бизнес-логики. *Vs Наблюдатель*: тот — субъект рассылает многим напрямую; посредник — центральный хаб. Обмен можно строить на «Наблюдателе». **[+]** шина событий, in-process медиатор (например, **Mediator** в .NET).

5. Паттерн «Команда» (Command)

Заворачивает действие в объект с методом **Execute()**. **Client** создаёт **Receiver** (бизнес-логика), **Invoker** (инициатор), **Command** (с ссылкой на **Receiver**); один объект команды раздается меню/кнопке/хоткею — они не знают, что она делает. По сути «Стратегия» для **Invoker**. Команды бывают «глупые» (зовут **Receiver**) и «умные» (оркеструют), есть составные (макросы, «Компоновщик»). **Undo/Redo**: два стека; **Execute()** + на **Undo**; отмена — **Undo()** + на **Redo**; новая команда чистит **Redo**; стеки держит **Controller**; мутабельную команду копируют («Прототип»); сочетается с «Хранителем» (команда — **Caretaker**). **[+]** **QAction** (Qt), команды **CQRS**, очереди задач; единое место для логирования/валидации.

6. Паттерн «Цепочка ответственности» (Chain of Responsibility)

Запрос идёт по *цепочке* обработчиков, пока кто-то не обработает. Базовый класс хранит ссылку на преемника и по умолчанию передаёт дальше; потомки переопределяют **HandleRequest()**: могут — обрабатывают, нет — зовут предка. Развязывает отправителя и получателя, цепочку можно менять динамически (плагины). Пример: контекстная справка — элемент, объемлющий элемент, и вверх по дереву. **Грбли**: цепочка обычно *уже есть* (дерево контролов); несколько видов запросов — представить запрос *объектом* (**BaseRequest** с наследниками, как **EventArgs**), а не плодить методы; *нет гарантии* обработки. **[+]** распространение исключений (**throw/catch**), всплытие событий (Qt/DOM), **middleware-конвейеры** (ASP.NET/Express), сервлет-фильтры.

7. Паттерн «Состояние» (State)

Context делегирует работу объекту-**State**; поведение меняется со сменой состояния, незаметно для клиента. Замена code smell «тэг типа» (переменная + `switch`) на иерархию состояний; имитирует «смену класса в рантайме». Пример: `TcpConnection` (`Established/Listen/Closed`) делегирует методы состоянию. *Vs Стратегия*: «Состояние» само переключается (паттерн определяет, как), «Стратегию» меняет клиент. **Габри**: переходы в **Context** (центрально, можно таблицей) или в **State** (гибче, но размазано); создание/удаление состояний (осторожно с `delete this`); разделяемые состояния — «Приспособленец». **[+]** реализация конечных автоматов вручную (ср. `switch`/таблица/паттерн «Состояние»); `XState`, `Stateless`; для скорости (лексеры) — таблицы переходов.

8. Паттерн «Посетитель» (Visitor)

Выносит операции из структуры объектов в отдельные классы. В узлах — одна операция `Accept(visitor)`; в **Visitor** — по методу `Visit` на тип узла. Механизм — **двойная диспетчеризация**: **Ассепт** (по типу узла) зовёт `Visit` (по типу посетителя). Пример: операции над **AST** (проверка типов, печать, кодогенерация) вместо *N* методов в каждом из десятков типов узлов. **Габри (проблема выражения)**: новую операцию добавить легко (новый `visitor`), новый *тип узла* — трудно (править все посетители); нарушает инкапсуляцию; `VisitClassA()` читабельнее перегруженного `Visit()`; обходом управляет структура, элемент или сам посетитель. **[+]** современная альтернатива — сопоставление с образцом по `sealed`-типам (`Java21`, `C#`, `Kotlin`, `Scala`); `visitor` как `map/fold`.

9. Паттерн «Хранитель» (Memento)

Сохраняет и восстанавливает состояние объекта, не раскрывая деталей посторонним. **Originator** делает слепок и восстанавливается; **Memento** — слепок; **Caretaker** хранит, но внутрь не лезет. **Memento** имеет «широкий» интерфейс для **Originator** и «узкий» для остальных. Пример: отмена в векторном редакторе — слепок прячет координаты фигуры и связей, команда (**Caretaker**) хранит его и при `Undo()` отдаёт сцене. **Габри**: нужна языковая поддержка узкого/широкого интерфейса (`friend` в `C++`, вложенный класс в `C#/Java`); слепок может быть *дельтой* (как системы контроля версий) — экономно, но хлопотно. **[+]** связка с «Командой» для `undo/redo`; сериализация, `event sourcing`, немутабельные снимки.

10. Паттерн «Интерпретатор» (Interpreter)

Лёгкая замена компилятору/интерпретатору: *абстрактное синтаксическое дерево* (по сути «Компоновщик») с операцией `interpret(context)`, которую каждый узел реализует сам. Пример: разобранное регулярное выражение — дерево с `interpret()`, проверяющим строку. **Когда**: грамматика *простая*, операций мало и они несложные, скорость не критична — маленькие встроенные **DSL** (*10-е правило Гринспена*: программы обрастают своим мини-языком). «Спецификация» — «Интерпретатор» для булевых предикатов. *Vs Посетитель*: много или сложные операции ⇒ «Посетитель»; «Интерпретатор» — только простая грамматика и немного простых операций. **[+]** внутренние/внешние **DSL**, деревья выражений (`LINQ Expression<T>`), парсер-комбинаторы.

11. Паттерн «Итератор» (Iterator)

Инкапсулирует способ обхода однородной коллекции, скрывая порядок и метод; клиент просит итератор у **Aggregate** и работает через интерфейс **Iterator**. Одну коллекцию можно обходить многими итераторами и разными способами; можно генерировать данные на ходу (бесконечные последовательности). **Два стиля**: `Java` (`hasNext/next/remove`) и `.NET/C++` (`MoveNext/Current/Reset`). **Внешний vs внутренний**: внешний — клиент сам шагает (гибко); внутренний — коллекции передают действие, она сама обходит (`ForEach`). **Габри**: курсор (логика обхода в коллекции); устойчивые vs *неустойчивые* (`fail-fast`) — большинство неустойчивы, ловят изменение по *версии* коллекции и бросают исключение. **[+]** прячется за `foreach`, генераторами (`yield`), **LINQ**/потоками; интерфейс `Iterable/IEnumerable<T>`.

Лекция 9. Архитектурные стили

Экзаменационные вопросы — краткие ответы

1. Понятие архитектурного стиля; трёхзвенная архитектура

Стиль — набор решений, которые: применимы в выбранном контексте, задают ограничения на архитектуру, дают желаемые положительные свойства. Задаётся *тремя* соображениями: набор **элементов**; **правила соединения** (топологические ограничения — *суть стиля*); **семантика** элементов.

Иерархия по масштабу и привязке к предметной области:

- *Паттерн* — тактика, уровень классов, вне предметной области;
- *Архитектурный шаблон* — типовое решение типовой задачи (MVC, Sense-Compute-Control);
- *Стиль* — уровень подсистем/системы, в предпочтительной предметной области;
- *Reference-архитектура* — предметно-ориентированная, обычно из стандартов.

Зачем стили: переиспользование архитектуры и кода (есть готовый middleware); общий словарь (назвал стиль — поняли); упрощение интеграции; специфичный анализ (граф у каналов и фильтров) и визуализация (Data Flow Diagram, DSL).

Трёхзвенная (State-Logic-Display): представление и взаимодействие с пользователем / бизнес-логика / слой хранения. **Ограничения:** клиент не знает БД и не лезет в неё напрямую; логика ничего не хранит и не делает UI; БД не делает нетривиального. Хорошо масштабируется: много клиентов на одну логику, много логики на одну (распределённую) БД.

Где: большинство веб-приложений и информационных систем, многопользовательские игры (клиент сложный, но не реализует игровую логику и не хранит данные, кроме UI).

vs MVC: трёхзвенка *специфицирует* хранение (есть слой данных); MVC — нет.

[+] реализуется библиотеками веб-разработки (готовое неизменяемое звено).

2. Шаблоны Model-View-Controller и Sense-Compute-Control

MVC — архитектурный шаблон (иногда классят как паттерн; у GoF с него начинается изложение).

- **View** — только отображение данных;
- **Controller** — логика взаимодействия: принимает ввод, командует Model, при нужде управляет View;
- **Model** — данные и, как правило, бизнес-логика; сама шлёт нотификацию об изменении.

Ограничения: View только читает Model; команды Model отдаёт только Controller; Model сама рассылает сигнал обновления ⇒ несколько View синхронно обновляются. Controller — единственная точка команд пользователя и естественное место Undo, Redo (паттерн «Команда»).

vs трёхзвенка: MVC не специфицирует, где хранить данные, но требует выделенного обработчика ввода (Controller). **Где:** десктоп с развитым UI (обычно лишь вершина айсберга над Model).

Sense-Compute-Control — шаблон робототехники и умных домов. Три фазы в бесконечном цикле: **Sense** (датчики) → **Compute** (вычисление воздействия; может включать SLAM и модель мира) → **Control** (актуаторы). Для простых систем решает всё; для сложных — лишь внешний каркас (поэтому шаблон, а не стиль).

[+] робототехнический middleware: ROS 1 (EOL май 2025) сменился на **ROS 2** (на DDS, publish-subscribe, без центрального мастера). Для UI-частей сегодня обычно **MVVM** (на iOS — макрос @observable).

3. Слоистый стиль; «Клиент-сервер»

Слоистый стиль: система делится на слои; каждый пользуется слоями ниже и даёт интерфейс выше, ничего о верхних не зная. «Многоуровневый клиент-сервер» (слой — клиент нижних и сервер верхних). Компоненты — слои; соединители — протоколы или интерфейсы. Бывает **строгим** (только непосредственно нижний слой) и **нестрогим** (любой ниже).

Плюсы: рост абстракции снизу вверх (программируешь в терминах слоя); лёгкое сопровождение (изменение влияет только выше); сменные реализации уровня при общем интерфейсе (Ethernet или WiFi; графика OpenGL, Vulkan, DirectX под движком).

Минусы: не всегда применим (осциллограф с первой лекции: данные идут вверх, команды вниз — все знают всех); оверхед прокидывания запросов при многих уровнях.

Примеры: трёхзвенка, сетевые стеки OSI и TCP/IP, ОС, DDD (представление / приложение / домен / инфраструктура).

Клиент-сервер — вырожденный слоистый (два уровня). Клиенты не общаются между собой, только с сервером; сервер не знает о клиентах (даже их число) до контакта. Серверов может быть много (масштаб, но синхронизация состояния). **Где:** веб, мобильные, ОС (графика Linux — оконный сервер).

[+] профстандарт «архитектор» подразумевает слоистую как архитектуру по умолчанию.

4. Гексагональная и луковая архитектуры

Гексагональная («порты и адаптеры») — развитие слоистой: инфраструктура тоже считается внешним миром. Центр — домен (бизнес-логика); вокруг — **порты** (предоставляемые и потребляемые интерфейсы) и **адаптеры** (валидация и преобразование данных). Механика: Dependency Inversion плюс Dependency Injection.

Два рода портов: primary (driving) — инициация действия, средства доставки запросов; secondary (driven) —

служебная функциональность, которой управляет ядро.

Плюсы: изоляция домена и от «механизмов доставки», и от инфраструктуры; лёгкое тестирование (внешние зависимости заменяются mock-заглушками); адаптеры берут на себя валидацию и конвертацию. **Минусы:** оверинжиниринг для простых систем; размытая грань «платформа или зависимость»; оставляет слишком много свободы. («Шестиугольник» — просто исторически так рисовали.)

Луковая (Onion) — уточнение гексагональной с уровнями внутри ядра: домен (центр) → доменные сервисы → сервисы приложений → внешний мир. Уровневость *нестрогая*. Репозитории логично держать снаружи (хотя в оригинале были в доменной модели).

vs DDD: структура близка, но DDD разделяет инфраструктуру и приложение, а доменные сервисы у DDD — полноценные жители домена (тут вынесены в отдельный слой).

[+] естественные спутники — CQRS (разделение моделей чтения и записи) и BFF.

5. Чистая архитектура (Clean)

Чистая архитектура — дальнейшее уточнение луковой и гексагональной. То же: слоистость, самый внутренний слой — доменная модель, зависимости строго снаружи внутрь («правило зависимостей»), Dependency Injection. **Сверх того** — специфицирует поток управления и конкретные роли взаимодействия с пользователем.

Поток в рантайме:

1. Запрос (сеть или GUI) попадает в **Controller** — точку входа;
2. Controller валидирует, парсит, конвертирует в **Request Model**;
3. Controller дёргает **Interactor** (известен по интерфейсу Boundary, подставлен через DI);
4. Interactor через **Entity Gateway** (репозиторий, тоже по интерфейсу) находит сущности;
5. Репозиторий грузит, создаёт или возвращает сущности — это *вне* границ системы;
6. Interactor координирует сущности и формирует **Response Model**;
7. Response Model уходит **Presenter** (известен Interactor только по интерфейсу);
8. Presenter строит **View Model** и отдаёт клиенту;
9. **View** отображает обновлённые данные.

Двойные полосы на схеме — границы компонента; сущности в самом центре, репозиторий снаружи.

Чем хорошо: чёткое разделение ответственности упрощает разработку и тестирование ⇒ стала очень популярной. **vs луковая, гексагональная:** добавляет именно поток управления и роли (Controller, Interactor, Presenter; модели Request, Response, View).

[+] VIPER (View-Interactor-Presenter-Entity-Routing) — буквальная Clean для iOS. Сегодня VIPER — территория UIKit; в SwiftUI обычно **MVVM** (@observable), а для сложного состояния — **TCA** (The Composable Architecture).

6. Пакетная обработка; каналы и фильтры

Пакетная обработка — «прадедушка стилей» (финансовые системы на магнитных лентах). Система — набор *отдельных программ*, выполняемых последовательно и обменивающихся данными средствами ОС (ленты → файлы, pipes, named pipes). Между процессами явно передаётся всё нужное: данные, конфигурация, команды.

Где: Linux way — мелкие утилиты, соединённые пайпами в конвейеры (скрипты, DevOps). **Плюс:** независимость программ (любой язык, переиспользование готовых). **Минус:** неторопливость (запуск процессов, межпроцессное общение).

Каналы и фильтры (pipes and filters): программа — набор **фильтров**, преобразующих данные в **каналах**. Фильтры независимы: нет разделяемого состояния, видят только свои входные каналы. Фильтры — единственные элементы, каналы — единственные соединители.

vs пакетная обработка: фильтр не обязан быть отдельной программой (нет проблем с производительностью); сети фильтров бывают сложными, не только линейными.

Варианты: конвейеры (линейная цепочка); ограниченные каналы (очередь конечной длины — против «пробок»); типизированные каналы (фильтр к каналу нужного типа — пример с осциллографом).

Плюсы: поведение = последовательное применение фильтров (легко рассуждать); лёгкая замена и переиспользование; анализ как граф (пропускная способность, задержки, дедлоки); параллелизм (фильтр — кандидат в поток, процесс, микросервис). **Минусы:** последовательное исполнение (простой при редких данных); интерактивность (данные идут в одну сторону, нужны обратные каналы); пропускная способность ограничена самым узким фильтром.

[+] современные движки потока и пакета: Apache Spark, Flink, Apache Beam, Airflow, GStreamer; визуальные DSL — LabVIEW, Matlab/Simulink.

7. Стиль Blackboard

Blackboard — стиль с *общей памятью*. Есть центральное хранилище («доска») и компоненты-источники знаний, которые знают только про доску, не имеют своего состояния и делают полезную работу. Каждый смотрит на доску, (возможно) находит данные, которые умеет преобразовать, преобразует и пишет обратно — в надежде, что подхватит кто-то ещё.

Метафора: эксперты у доски, каждый шарит в своём. Порядок действий навязывается только через *токен испол-*

нения, хранимый на доске.

Плюсы: компоненты независимы, работают параллельно, разрабатываются отдельно; лёгкие масштаб и расширение (добавил компонент); не нужно даже знать алгоритм решения («бросаешь компоненты, пока не решится»).

Минусы: задача легко может не решиться (потому стиль и применяют в ИИ, где задача и так бывает нерешаема); доска — узкое место (синхронизирует доступ независимых компонентов).

Родственник: стили на правилах переписывания (машины Маркова, язык Рефал, графовые грамматики) — тоже центральная структура данных плюс правила, независимо её меняющие.

[+] идея доски живёт в пространствах кортежей (Linda, JavaSpaces) и общих хранилищах (Redis как разделяемое состояние); логику правил дают rule engine (Drools). Современный всплеск — мульти-агентные LLM-системы с общим «scratchpad».

8. Событийно-ориентированные стили; Publish-Subscribe; событийная шина

Событийный стиль (неявный вызов): вместо явных вызовов — оповещения. Есть слушатели, подписывающиеся на события; при наступлении события система сама вызывает зарегистрированных. У компонентов два вида интерфейсов — методы и события. **Инварианты:** источник не знает, кто и как отреагирует (обычно не вправе узнать даже число подписчиков); нет предположений, будет ли событие обработано.

Плюсы: низкая связность $\Rightarrow$ переиспользуемость; лёгкое конфигурирование (меняешь подписки — меняешь поведение). **Минусы:** неинтуитивная структура (клубок подписок); компоненты не управляют порядком вычислений; неизвестно, кто и когда ответит (принципиальная асинхронность); тяжёлая отладка; гонки даже в одном потоке (результат зависит от порядка вызова обработчиков); опасен антипаттерн «Sequential coupling».

Publish-Subscribe: издатели публикуют сообщения (синхронно или асинхронно), подписчики подписываются и получают события, маршрутизаторы (router) — посредники (фильтрация, балансировка нагрузки). Соединители — сетевые протоколы, очереди сообщений или (на одной машине) паттерн «Наблюдатель». Издатели не знают подписчиков; подписчики обычно не знают друг друга.

Событийная шина: компоненты не общаются напрямую — только через шину (подписаться можно лишь на события в шине, посылать — только шине). Два режима: **push** (шина уведомляет сама) и **pull** (компоненты опрашивают шину). Шин может быть несколько именованных. Хороша для распределённых систем. Приём **Event Sourcing**: вместо явного состояния — append-only лог изменений; даёт eventual consistency (теорема CAP), аудит, откат, снапшоты. Уточнение — **Enterprise Service Bus**: умная шина с преобразованием данных, интеграция сторонних приложений.

vs: Observer (на машине) $\rightarrow$ Pub-Sub (брокер или сеть) $\rightarrow$ шина (только через шину, push/pull).

[+] современные брокеры и потоки: Apache Kafka (в 4.x — нативные очереди), RabbitMQ, NATS, облачные Pub/Sub, Azure Service Bus. ESB сегодня — тяжёлое легаси («умные конечные точки, тупые трубы»); Event Sourcing идёт в паре с CQRS; в P2P роль шины играет GossipSub.

9. Peer-to-peer

Peer-to-peer (одноранговая сеть): система из множества приложений, каждое из которых в принципе самодостаточно (своё состояние, свой поток управления), но чем больше узлов работают совместно, тем шире возможности. Компоненты — отдельные приложения; соединители — сетевые протоколы; данные — сообщения по сети.

Топология: никаких ограничений на связи (более того, приветствуются избыточные связи); топология динамична — узлы выходят и появляются во время работы.

Плюсы: отказоустойчивость (можно физически уничтожить часть — остальное продолжит работать, менее эффективно); лёгкий масштаб динамическим подключением узлов $\Rightarrow$ идеален для распределённых вычислений, где участник «никому ничего не должен». **Минус:** сложность координации и согласования состояния (та же теорема CAP).

Примеры: BitTorrent (каждый узел — файловый сервер; больше узлов — больше данных и быстрее скачивание); сенсорные сети, стаи беспилотников (при потере узла стая переконфигурируется и продолжает задание).

[+] современный фундамент P2P — **libp2p** (выделился из IPFS): стек поверх TCP, QUIC, WebRTC с обнаружением через DHT (Kademlia), NAT-обходом и pub-sub (GossipSub); на нём построены Ethereum, IPFS, Filecoin, Celestia. Прочие примеры — блокчейны (Bitcoin), WebRTC, Syncthing, Matrix.

Лекция 10. Предметно-ориентированное проектирование

Экзаменационные вопросы — краткие ответы (DDD)

1. Понятие DDD; единый язык

Domain-Driven Design — методология, строящая архитектуру вокруг «смыслового ядра» — **модели предметной области**, выраженной *прежде всего в коде* (сущности области → классы), а также в диаграммах и устной речи. Три опоры: **переработка знаний** (строим модель, одновременно изучая область, пока не поймём её *принципы*); **единый язык**; **устное общение** (неуклюжие имена слышны в разговоре; естественный язык подсказывает решения — архитектура про понимание *человеком*).

Единый язык (ubiquitous language) — один язык команды и экспертов, используемый *повсюду* (ТЗ, классы, методы, переменные, тесты). Синонимы запрещены: термин закреплён как имя класса — только оно. **Что входит:** термины области (классы); действия и отношения (методы); имена паттернов, даже если их в области нет («репозиторий», «спецификация»); высокоуровневые понятия архитектуры, не выражимые в коде (уровни, «канал»).

Эволюция: язык меняется вместе с пониманием ⇒ за изменениями в речи следят и рефакторят код («связь» вместо «ассоциация» → переименовать класс).

Несколько языков: у обособленных частей могут быть свои языки и модели («перевозка» в системе vs «ребро» в библиотеке графов) — *если* есть чёткая граница и правила преобразования (фасады, адаптеры). Иначе модели проникают друг в друга ⇒ путаница.

[+] это стратегический DDD: **Ограниченный контекст** (Bounded Context, зона однозначного языка), **Карта контекстов** (Context Map), **антикоррупционный слой**. Контекст сегодня часто = микросервис или модуль монолита. Язык и границы ищут на **Event Storming** (Брандолини: стикеры доменных событий на таймлайне) и Domain Storytelling — лёгкая замена UML.

2. Изоляция предметной области; антипаттерн «Умный GUI»

Идея: чтобы код модели был маленьким, простым и содержал только суть, всё прочее (UI, сеть, БД, утилиты, логи, тесты) архитектурно *отделяют* от модели; прочие модули просто *используют* её классы.

Способ — уровневая архитектура: уровень общается напрямую только с уровнями ниже. DDD не диктует разбиение, требует лишь, чтобы *был* уровень предметной области. Типично четыре уровня:

- **Интерфейс** — только отображение и обработка событий, без логики;
- **Операционный** (application) — тонкий, координирует, инициирует операции; *без состояния* (кроме связанного с UI, типа прогресса); специфичен для приложения;
- **Предметной области** — бизнес-объекты и правила (и регламенты процессов); переиспользуем в семействе; здесь конкурентное преимущество;
- **Инфраструктурный** — БД, сеть, ORM, каркасы; поддерживает всех выше (деревья наследования растут вверх — базовые классы могут быть тут).

Вниз — напрямую, вверх — косвенно: callback-и (hook-и), «Наблюдатель», MVC. Труднее отлаживать, зато уровни независимы.

Антипаттерн «Умный GUI»: вся бизнес-логика (и работа с БД) в обработчиках событий GUI; доменных классов нет. **Ок** до ~3000 строк (RAD-тулзы генерят, быстро, фичу добавить легко, код прост). **Зло** дальше: нет проектирования по модели, правило для двух фич пишется *дважды*, сложное поведение не декомпозировать, интеграция/API почти невозможны.

[+] современная изоляция — гексагональная / луковая / чистая архитектура (порты и адаптеры, домен в центре). Границы проверяют автоматически: jMolecules (@DomainLayer и т.д.) + ArchUnit. «Умный GUI» сегодня — low-code / no-code платформы.

3. Структурные элементы модели; идентичность объекта

Ассоциации: по возможности *однаправленные* (двунаправленная мешает понять объект отдельно); минимизировать множественность (квалификатор «страна → год → президент») и число связей.

Сущность (Entity) — собственная *идентичность*, не зависящая от состояния. **Объект-значение** (Value Object) — определяется *только атрибутами*. Похоже на ссылочные типы vs типы-значения, но ссылочная идентичность не переживает сериализации.

Идентичность в ИС сложна: должна переживать save / load / передачу в другую систему и иной формат ⇒ нужна явная **операция идентификации** (ссылочное равенство не годится). Естественной идентичности почти нет (паспорт меняют, ИНН / СНИЛС не у всех, тёзки). Звонок клиента: старый со знакомого номера / старый с новым номером / новый со старым номером — номер обманчив. **Суррогат** (GUID; иногда виден — трек-номер, иногда нет — id в Google Docs) vs **естественный атрибут** (номер кресла; сам решает проблему дубликатов, но мнимая уникальность опасна).

Всё возможное — в значения (немутабельные, единое целое: «имя + фамилия» — один VO; могут быть ссылочными для разделяемости). **Сущность-или-значение — решение проектирования:** билет с местом → место сущность; без места → значение.

Службы — классы без состояния (операция над несколькими *равноправными* объектами: диспетчеры, Маршрутизатор); зовут откуда угодно в любом порядке. **Модули** — группировка по *семантике* (единый язык), не по метрикам; cohesion / coupling получаются сами.

[+] современный кирпичик — **доменные события**; немутабельные значения пишут как record (Java / C#) или data class (Kotlin); кирпичики размечают jMolecules (@Entity, @ValueObject, @Identity).

4. Паттерн «Агрегат»

Агрегат — изолированный кусок модели с **корнем** и **границей**. Доступ ко всем элементам — *только через корень*. Корень — сущность с *глобальной* идентичностью; внутренние объекты — *локальная* идентичность (в рамках агрегата). Единица инкапсуляции: внутри думаем о связях, снаружи — только о корне.

Зачем: упрощает и декомпозирует модель; технически — транзакции к *разным* агрегатам не мешают друг другу (легче обновления в БД). **Инварианты** держит корень. Содержимое может быть видно снаружи, ссылки на внутренние объекты отдавать можно, но *хранить* извне нельзя ⇒ отдавать немутабельные или копии.

Пример: автомобиль — корень; колёса и шины внутри (важно, на каком колесе какая шина, но глобальная идентичность шины не нужна — локальной хватает); двигатель — часть машины, но имеет *свой номер* (глобально уникален).

[+] правила Вернона (red book): агрегаты *маленькие*; ссылка на другой агрегат — *только по id*; одна транзакция — *один агрегат*, согласованность между ними — *отложенная* через доменные события. Граница агрегата = транзакции = согласованности (часто = микросервиса). Spring Modulith проверяет это на сборке.

5. Паттерны «Фабрика», «Репозиторий»

Фабрика — создаёт и инициализирует агрегат (или объект) так, чтобы выполнялись *инварианты*. Инкапсулирует сложное создание, не раскрывая структуру, избавляет агрегат от обязанности создавать себя. Создание *атомарно* (сбой → клиент не получает ничего). Реализуется через Фабричный метод, Абстрактную фабрику, Builder. Умеет и *восстанавливать* объект из хранилища (разбор XML) — *не меняя идентичность*. В области явно почти не присутствует, но часть модели и языка.

Репозиторий (Хранилище) — хранение, сохранение, загрузка; обычно *глобальный доступ*. Прослойка между приложением и БД, может держать объекты в памяти. Нужен, чтобы не искать объект длинной цепочкой ссылок от корня (это блокировало бы половину БД); лучше хранить *id* и брать из репозитория. Имеет **язык запросов** (по *id* или выборка по атрибутам / спецификации); за интерфейсом прячет реальный способ (ORM + БД, объектная БД, фабрики). Отлично дружит с агрегатами: корень по глобальному *id* + сборка остального.

Фабрика ≠ репозиторий: фабрика делает *новый* объект, репозиторий находит *существующий* (часто через фабрику для восстановления).

[+] репозиторий — самый выживший паттерн: Spring Data (JpaRepository, производные запросы), EF Core (DbContext = репозиторий + Unit of Work), jMolecules @Repository. Фабрики — это билдеры и DI-контейнеры; восстановление — гидрация ORM. При CQRS читающий и пишущий репозитории разделяют.

6. Паттерн «Спецификация»

Контекст: моделирование **ограничений** на состояние; ограничения умеют порождать новые объекты, которых будто не было в области. Лестница: проверка вперемешку с кодом → *метод с говорящим именем* → *отдельный класс* для сложных ограничений.

Спецификация — класс, проверяющий объект на соответствие ограничению; булев метод isSatisfiedBy(o). Сложные спецификации собирают из простых через «И», «ИЛИ», «НЕ» (паттерн **Компоновщик**).

Три применения: (1) *проверка* ограничения (большой assert); (2) *выборка* объекта из репозитория (по сути упрощённое дерево разбора SQL); (3) *построение* объекта в фабрике («хочу однонаправленный граф с примерно равным числом вершин и рёбер» — фабрика выберет реализацию).

Пример: склад химикатов; контейнер даёт набор характеристик, требование химиката — спецификация (isSatisfiedBy проверяет наличие нужной характеристики у контейнера), контейнер проверяет isSafelyPacked.

[+] спецификация жива во фреймворках доступа к данным: Spring Data Specification<T> (типобезопасные критерии, комбинируются and / or), LINQ и предикаты EF Core; по сути «объект-запрос» и функциональный Predicate<T>. Та же идея — в движках правил (rule engines).

Лекция 11. DDD: стратегические аспекты

Экзаменационные вопросы — краткие ответы

1. Ограниченный контекст; непрерывная интеграция; карта контекстов

Проблема: над большим проектом работают много команд с *разным видением*. Крайности плохи: держать модель цельной (неподъёмные затраты, выйдет слишком общей) или разрешить фрагментацию (ломает переиспользование и интеграцию). Пример: один класс «Платёж» на две команды с разным смыслом → падение на налоговых отчётах; решение — две абстракции («Платёж поставщику» / «Платёж клиента»).

Ограниченный контекст (Bounded Context) — кусок области и кода, где применима одна модель и единый язык. Внутри всё ей следует; снаружи не вправе предполагать о ней или переиспользовать без спроса. «Атомарная» область одной команды; разбиение следует *организационной* структуре (а та — высокоуровневой структуре системы).

Непрерывная интеграция (внутри контекста): слияние в основную ветку раз в несколько часов, сборка + юнит-тесты после каждого коммита, высокое покрытие, постоянное общение. 3–4 человека легко договариваются, но дробить систему на такие контексты непрактично — CI держит единство в более крупном.

Карта контекстов (Context Map) — фиксирует, как понятие одной модели транслируется в понятие другой (естественный язык или диаграмма). Пример: контекст «прокладка маршрута» (Route Spec, Route, Leg) и контекст «графы» (узлы, дуги, веса) общаются через сервис-переводчик; Leg и Arc — одно понятие, дубликация осмысленна (параллельная работа).

[+] «контекст следует за орг-структурой» — закон Конвея (1968); развитие — Team Topologies (2019): контекст ↔ stream-aligned-команда, обратный манёвр Конвея, Team API ≈ Open Host Service, когнитивная нагрузка. Карты — DSL Context Mapper; стыки — контрактные тесты (Pact).

2. Подходы к интеграции контекстов

Степень интеграции зависит от обстоятельств; высокая нужна не всегда (один из паттернов — «не интегрироваться вовсе»). Семь паттернов Эванса:

- **Общее ядро** (Shared Kernel) — совместная реализация общей части; команды общаются *прямо*, на одном уровне; ядро меняют только по согласованию обеих;
- **Заказчик-поставщик** (Customer-Supplier) — общение затруднено или есть зависимость, разные циклы релизов; явные роли, заказчик даёт задачи + приёмочные тесты; одна иерархия управления;
- **Конформист** (Conformist) — полная зависимость от компонента, на который не повлиять (legacy, платформа); принять чужую модель как есть (вопреки «not invented here»); часто мудро;
- **Предохранительный уровень** (Anticorruption Layer) — чужая модель противна, но компонент нужен; прослойка-переводчик понятий (Фасад, Адаптер, службы); много кода;
- **Раздельное существование** (Separate Ways) — выгода < затрат; никакой интеграции;
- **Служба с открытым протоколом** (Open Host Service) — у поставщика *много* заказчиков; строим модель сами, публикуем интерфейс + документацию, заказчики подстраиваются;
- **Общедоступный язык** (Published Language) — много заказчиков *и* поставщиков; совместно разработать, стандартизовать и навязать общую (простую, выразительную) модель.

Сравнение: справа-вверху максимум интеграции (единый контекст, Общее ядро); ниже — Заказчик-поставщик и Открытый протокол; слева-внизу «когда всё плохо» (Предохранительный уровень, Конформист, Раздельное существование). Пример «слон и слепцы»: Раздельное → ACL → Общее ядро → единый контекст.

[+] **Общедоступный язык** — версионизируемые контракты (OpenAPI / AsyncAPI / protobuf / GraphQL); **Открытый протокол** — публичный API за шлюзом; **Предохранительный уровень** — Strangler Fig (Файлер) + ACL при переезде с legacy (паттерны Microsoft и AWS). Добавляют Партнёрство и анти-паттерн Big Ball of Mud.

3. Смысловое ядро; приёмы дистилляции; абстрактное ядро

Смысловое ядро (Core Domain) — самая содержательная часть, *know-how*, за что платят и чем продукт отличается; всё прочее можно брать из open source. Ядро должно быть *самым проработанным, маленьким и чётко отделённым*. Ирония: ядром не любят заниматься (требуется погружения в область, в резюме «не звучит»), но специализация в области ценнее тающего стека технологий.

Дистилляция — выделение ядра и вынос лишнего:

- **Domain Vision Statement** — ~1 страница: что ядро умеет и зачем (не архитектура);
- **Выделенное ядро** (Highlighted Core): *дистилляционный документ* (3–7 страниц о составе и взаимодействии) или *Flagged Core* (пометить элементы стереотипом / цветом);
- вынести неспецифичное в модули: **Неспециализированные подобласти** (Generic Subdomains — важны для области, не для *этой* системы) и **Связные механизмы** (Cohesive Mechanism — неспецифичны для области вообще, например графы → берём библиотеку).

Пример: ядро логистики — Cargo, RouteSpecification, Itinerary, HandlingStep, BillOfLading; не ядро — Customer (это CRM), прокладка маршрута (библиотека); сложность ядра вдвое меньше.

Абстрактное ядро: если ядро велико — оставить в нём *только абстрактные классы / интерфейсы*, реализации

вынести в модули. Сложность радикально не снизит (essential complexity), но уберёт детали.

[+] классификация подобластей (Бернон): Core / Supporting / Generic; стратегия buy-vs-build (ядро пишем сами, обобщённое — Stripe, Auth0, CRM-SaaS). Flagged Core — стереотипы jMolecules + ArchUnit.

4. Крупномасштабная структура; метафора системы; разбиение по уровням

Крупномасштабная структура (Large-Scale Structure) — самая «архитектурная» часть: основные правила для всей системы; обычно *уточнённый под систему стиль* + специфичные ограничения (главные компоненты и их ответственности, правила добавления новых). *Эволюционирует* с переработкой знаний; не фиксировать навсегда, не делать жёсткой, не насаждать административно (антипаттерн «Архитектор в башне из слоновой кости»). Маленькие проекты живут без неё; носитель — *единый язык* и устная традиция, не UML-фолиант (автогенерация диаграмм — не архитектура).

Метафора системы — аналогия для понимания целого (рабочий стол, файервол, дерево). Опасность — тащит лишний смысл (файервол → забыли про локальный трафик; стол не бесконечен). Хорошая метафора (из области) сильно помогает.

Осмысленные уровни: нельзя делить механической топосортировкой графа зависимостей; разбиение должно нести смысл из области и *объяснять* (качество архитектуры = объясняющая способность). Пример: грузоперевозки делят на Ресурсный / Операционный / Поддержки решений; топологическое правило (низ не знает о верхе) держат «Репозиториум». Опасные грузы: *неверно* — Груз зовёт уровень выше; *верно* — спецификацию получает Маршрутизатор (переработка знаний). **Мораль:** структура приоритетнее сиюминутных требований, ради неё рефакторят (иначе эрозия, Lava Flow).

[+] документируют C4-моделью, ADR, arc42; соблюдение — фитнес-функции / ArchUnit («архитектура как код»). Осмысленные границы ≈ «fracture planes» из Team Topologies.

5. Типичные уровни в производственных и финансовых системах

Производство — четыре уровня (снизу вверх):

- **Потенциальный** — возможности (оборудование, площади, контракты); меняется *редко*;
- **Операционный** — оперативное управление (склад, статус, план); меняется *постоянно*;
- **Регламентный** — ограничения (законы, регламенты, стратегия); меняется *раз в годы*;
- **Принятия решений** — стратегия, анализ, оптимизация; своего состояния, как правило, *нет* (пользуется данными ниже).

Разный темп изменения полезен для проектирования схемы БД и оптимизации запросов.

Финансы — тоже четыре, но иначе: «Потенциального» нет (особым оборудованием не управляют). Самый нижний — **Операционный** (финансовые операции); выше — **Обязательства** (то, чем реально торгуют: договора, сделки; кредит = *обмен обязательствами*, а не «продажа денег»); **Регламентный** и **Принятия решений** — как в производстве.

[+] это предметные reference-архитектуры. Сегодня регламентный уровень часто реализуют движком правил (Drools) или DMN; уровень решений — аналитика / BI / ML; «разный темп изменения» — это rate-of-change-разделение (конфигурация vs данные); регламентный уровень нередко строят как «Уровень знаний».

6. Стили «Уровень знаний», «Подключаемые компоненты»

Кроме уровневой, бывают другие высокоуровневые структуры. Структура у системы *одна*, но стили комбинируются (уровневая в целом + «Уровень знаний» для одного уровня + плагины для другого).

Подключаемые компоненты (Pluggable Component Framework) — *ядро* + плагиновая система, динамически подключающая плагины через чётко определённые интерфейсы; плагины реализуют большую часть функциональности. Примеры: VS Code, отчасти IDEA, браузеры, Linux (ядро + модули).

Уровень знаний (Knowledge Level) — система содержит *данные* операционного уровня и *метаданные* (данные о данных) на уровне знаний; объекты уровня знаний определяют, *что и как можно делать* с операционными. Аналогия — рефлексия (Type / Class на уровне знаний, объекты на операционном). Можно *редактировать в рантайме* ⇒ операционный ведёт себя иначе (визуальное метамоделирование; финансовые продукты меняет даже не-программист). *Минус:* логика не проверяется компилятором (вся система как правила в XML → кошмар поддержки).

[+] Подключаемые компоненты — микроядро / плагиновая архитектура (расширения VS Code, OSGi, Eclipse RCP, Backstage). Уровень знаний — движки правил (Drools), BPMN / DMN, feature flags (LaunchDarkly), low-code / no-code, конфигурация-как-данные. «Не-программист меняет правила» — сегодняшний no-code; минус тот же (нет проверки типов, ошибки в рантайме).

Лекция 12. Распределённые приложения: технические вопросы

Экзаменационные вопросы — краткие ответы

1. Понятие распределённой системы; заблуждения при проектировании

Распределённая система — приложение, компоненты которого находятся в сети и взаимодействуют через обмен сообщениями, работая с общими ресурсами (доступ многим пользователям / с разных устройств, распределённая обработка и хранение).

Принципиальные отличия: (1) *многопроцессная* параллельность — нет общей памяти, дорогая коммуникация; (2) *независимые отказы* компонентов + ненадёжная сеть (временные отказы); (3) *нет единого времени* — пакеты приходят в разном порядке / в разное время / не приходят (это норма) ⇒ гонки большого масштаба.

8 заблуждений распределённых вычислений (Дойч, Sun): сеть надёжна; задержка нулевая; пропускная способность бесконечна; сеть защищена; топология не меняется; администратор один; передача данных бесплатна; сеть однородна. Каждое неверно — сетевое приложение должно восстанавливаться после сбоев, не гонять лишнего, не считать сеть бесплатной и однородной.

Организация связи: межпроцессная (пайпы, если на одной машине), удалённый вызов (запрос-ответ, RPC, RMI), неявная (очереди, шины — вызывающий не знает вызываемого). Размещение — сервисы по серверам (+ балансировщики, мониторинг); **мобильный код** (как Google Docs) и **мобильный агент** (P2P, IoT); **кеширование** (Redis).

[+] заблуждения — канон. Средства борьбы: экспоненциальная выдержка + jitter, circuit breaker, таймауты (Polly / resilience4j), ключи идемпотентности; сквозное — service mesh (Istio, Linkerd), OpenTelemetry.

2. RPC, RMI. Пример: gRPC

RPC (Remote Procedure Call) — удалённый вызов как локальный. Программист пишет клиент, реализации процедур и языконезависимое описание интерфейса; по нему генерируются **заглушки** (stubs): клиентская берёт на себя соединение, сериализацию, передачу; серверная — десериализует, вызывает метод, сериализует ответ. **RMI** — тот же RPC + ООП (исключения, распределённые ссылки на объекты). RPC легче ⇒ микросервисы.

Прозрачность под вопросом: различать локальные и удалённые вызовы надо — удалённые уязвимы к отказам (сервиса → диагностика; сети → обработать, *exponential backoff*) и не мгновенны (асинхронно, с отменой). Семантика: *at least once* / *at most once* / *exactly once* (последняя — труднее всего).

protobuf — бинарный формат от Google (~10× компактнее XML), нужна схема (.proto); protoc генерирует код; у поля есть *номер* (эволюция, обратная совместимость); умеет enum, repeated, oneof. **gRPC** — поверх protobuf, описывает сервис; 4 режима: *unary* (запрос-ответ), *server streaming*, *client streaming*, *bidirectional*. Ответы — через StreamObserver (Наблюдатель: onNext / onCompleted / onError). Клиент — синхронная и асинхронная заглушки.

[+] gRPC — рекомендованная Microsoft замена WCF, поверх HTTP/2; для браузеров — gRPC-Web / Connect; в C# — grpc-dotnet (старый Grpc.Core EOL); у protobuf теперь editions. Конкуренты: Thrift, Cap'n Proto, FlatBuffers, Avro.

3. Веб-сервисы, SOAP. WCF

Веб-сервис — отдельная система с сетевым интерфейсом по стандартному протоколу (стиль RPC / RMI), *языконезависимая*. Технически — «запрос-ответ» поверх HTTP, человекочитаемая сериализация (XML / JSON), асинхронно, независимые отказы, часто очереди между сервисами.

SOAP (из XML-RPC, 1998) — передача XML-сообщений поверх транспорта (обычно HTTP, можно SMTP / FTP). Сообщение — **конверт** (Envelope) с **Header** (метаинформация) и **Body**. Стек: SOAP + **WSDL** (XML-описание возможностей, аналог .proto, стандарт W3C) + **UDDI** (реестр сервисов — умер в начале 2010-х).

+: генерация заглушек по WSDL и WSDL по коду; валидация в три слоя (парсер XML, XSD, сервис); человекочитаем, стандартен. **-**: огромный размер; без инструментов WSDL писать тяжело; нет транзакций; модель беднее стримов gRPC; тяжело мигрировать клиенты (решают версиями на эндпойнтах).

WCF (Windows Communication Foundation) — фреймворк .NET для веб-сервисов; **ABC: Address** (эндпойнт), **Binding** (транспорт, сериализация, безопасность), **Contract** (исполнение). Контракт размечают атрибутами (ServiceContract, OperationContract, DataContract); по ним генерится WSDL. Хост — ServiceHost; клиент — svcutil + app.config.

[+] WCF устарел с .NET 5, нет в современном .NET (жив на .NET Framework 4.8 для legacy); CoreWCF (2022) — лишь мост для миграции SOAP. Замена — gRPC или ASP.NET Core Web API; SOAP жив в банках / госсекторе.

4. Веб-сервисы, REST. ASP.NET Web APIs

REST (Representational State Transfer, Fielding, 2000) — представление состояния передаётся сервер↔клиент и *полностью* определяет состояние протокола; сервер **stateless** (не хранит клиентскую сессию) — архитектурно конечный автомат. Сессию хранит клиент ⇒ сервису всё равно, с какой машины запрос.

Использует семантику HTTP; запросы просты (часто GET из браузера), параметры — в URL, результат — код HTTP, тело — обычно JSON. Statelessness + URL ⇒ **кеширование** (важно для микросервисов). Всё вокруг **ресурсов** и **коллекций**; URL — коллекция / id, метод — действие: **GET** (список / элемент), **POST** (создать в коллекции), **PUT/PATCH** (обновить целиком / частично), **DELETE** (удалить). Коды: 200 ок, 201 создан, 204 без тела, 404 не найдено, 400 неверный, 409 конфликт.

ASP.NET Web APIs — серверный фреймворк .NET для REST. **Минимальные API**: MapGet отображает роут на лямбду (сервис в одном файле). Контроллер — только *средство доставки* (принять, десериализовать, передать

домену через адаптеры), не дёргать DAL напрямую. Различать модели: представления (DTO), доменную, персистентности.

[+] REST — стандарт де-факто для публичных API. Контракт — OpenAPI (бывший Swagger, встроен в ASP.NET с .NET 9). Альтернативы: GraphQL (один эндпойнт, клиент выбирает поля), gRPC (внутренние вызовы), tRPC. Транспорт — HTTP/2 и HTTP/3 (QUIC).

5. Очереди сообщений. RabbitMQ, Apache Kafka

Очередь сообщений — прослойка между клиентом и сервером с *гарантированной доставкой*, даже если они онлайн в разное время: отправитель кладёт сообщение в очередь, получатель забирает, когда готов. Развязка по времени; много отправителей / получателей («издатель-подписчик», шина событий); маршрутизация, фильтрация, преобразование.

RabbitMQ — отдельный сервер (Erlang), протокол **AMQP**. Сообщение публикуется в **exchange** (маршрутизатор; по умолчанию шлёт в одноимённую очередь), типы: direct, topic, fanout, headers. Получатель забирает и (обычно) удаляет сообщение. Параметры очереди: durable, exclusive, autoDelete.

Apache Kafka — не очередь, а **распределённый лог событий**: сообщение *записывается* и хранится (по умолчанию ~7 дней, можно вечно), а не вычитывается-и-удаляется. Подписчик читает лог последовательно и хранит **offset** (можно *отмотать* и переиграть). Архитектура: **брокеры** → **топики** → **разделы** (ключ → раздел по хешу). «Группа подписчиков» — событие одному участнику (балансировка). Немутабельный лог + распределённость ⇒ скорость и **Event Sourcing** (IoT, потоки). RabbitMQ — для надёжной доставки одному (событий немного), Kafka — для огромных потоков и переигрывания.

[+] Kafka 4.0 (2025) убрала ZooKeeper — координация через встроенный **KRaft**; добавлены Share Groups (Queues for Kafka). RabbitMQ 4.x — нативные streams, AMQP 1.0, MQTT 5.0; системы сближаются. Рядом — schema registry (Avro), dead-letter-очереди, паттерн Outbox.

Лекция 13. Распределённые приложения: архитектура

Экзаменационные вопросы — краткие ответы

1. Архитектурные стили: Big Compute, Big Data

Стили — подвиды слоистого или событийного; ниже по возрастанию «обычности» для облака.

Big Compute (большие вычисления) — для массовой параллельности (физические расчёты, симуляция, дифуры). Архитектура: клиент → очередь задач → **координатор** (делит задачу на подзадачи) → вычислительные узлы (десятки–десятки тысяч) → агрегация. Нужна *embarrassingly parallel* задача (в идеале Map-Reduce): узлы считают без координации (мощный CPU, слабая сеть); если нужна координация — высокопроизводительная сеть (кластер). Дорого и специфично, но иначе часто не решить.

Big Data (большие данные) — обратная сторона: вычисления просты, данных очень много (не лезут в СУБД). Импорт из источников, далее *на лету* (поток → очередь Kafka как буфер + балансировщик → потоковые обработчики → аналитическое хранилище) или *пакетно* (распределённое хранилище → периодическая обработка). Готовое: **Hadoop** (HDFS + MapReduce, пакетно), **Spark** (в памяти, быстрее). **Schema-on-read** — хранить как пришло, преобразовывать при чтении («Data Lake»; антипод «Data Swamp»); вместо ETL — **TEL**; личные данные удалять раньше. Пример — IoT.

[+] Hadoop уступил Spark (Databricks) и Flink (стриминг); вместо «озеро vs склад» — **Data Lakehouse** (Iceberg, Delta Lake, Trino) с ACID на озере.

2. Web-queue-worker, N-звенная архитектура

Web-queue-worker — уточнение событийного стиля для *вычислительно сложных* задач при несложной области (конвертация PDF). Клиент (браузер) → веб-приложение → очередь → обработчики → БД (через кеш); + внешние сервисы и CDN. Очередь и обработчик *не обязательны* (без вычислений — трёхзвенка), но очередь — буфер пиков + балансировщик, а разделение веб / обработчик даёт независимое масштабирование. Запросы асинхронны. Готовый стек собирается за выходные; минус — разрастается в «ком грязи».

N-звенная — браузер → веб-слой → бизнес-логика → слой данных (+ внешние сервисы, кеш на чтение). Компоненты на *разных машинах*; каждый слой — пул за **балансировщиком** (производительность, надёжность, авто-масштаб по метрикам). Когда брать: перенос монолита в облако с минимумом усилий; несложная бизнес-логика, требующая структуры. Минусы: масштаб только *крупными кусками*; бизнес-слой пухнет в «ком грязи». На Azure: балансировщик → NVA в DMZ → виртуальная сеть (веб / бизнес / СУБД), доступ через Bastion.

[+] Web-queue-worker часто бессерверный (FaaS / Lambda + managed-очередь SQS); N-звенная в облаке — контейнеры за managed-балансировщиком или PaaS; жива для lift-and-shift монолитов.

3. Микросервисная архитектура

Приложение делится на маленькие *отдельные веб-сервисы*; каждый — размером «одна небольшая команда», со своей моделью предметной области (это **ограниченный контекст** из DDD). Клиент → **API Gateway** (единый фасад, без бизнес-логики) → микросервисы; **оркестратор** (Kubernetes, Docker Compose) управляет жизненным циклом, масштабом, балансировкой, мониторингом (сложность — в связях, не в самих сервисах).

Плюсы: свой язык / стек у каждого; *своя БД* (не делит схему — как объект ООП: состояние + поведение через интерфейс); независимые *масштаб* (оркестратором, в контейнерах — дёшево), *деплой* (релизы не согласуют → несколько в день), *отказ* (локализуется); маленькая кодовая база. **Минусы:** сложность переходит с *кода на оркестрацию* (инструменты незрелы); неочевидные зависимости, нужна культура DevOps; приложение сложнее эквивалента; требовательно к сети (*нельзя сделать каждый класс микросервисом*); целостность данных не гарантируют — **eventual consistency**.

[+] маятник качнулся к **модульному монолиту** (CNCF 2025: ~42 % вернули часть сервисов; порог выгоды ~50 инженеров; откат Amazon Prime Video). Инфраструктура: service mesh (Istio, Linkerd), API-шлюзы (Kong, Envoy), BFF, sidecar.

4. Хорошие практики проектирования REST-интерфейсов

REST любим микросервисами (простая коммуникация, малая нагрузка на сеть). Рекомендации:

- вокруг **ресурсов**, а не действий (/customers/ хорошо, /get_customer/ плохо — действие это HTTP-метод);
- отношения — ресурсы в URL (/customers/5/orders), но без перебора уровней;
- API — модель *предметной области*, не данных (не раскрывать таблицы БД);
- стандартная семантика HTTP — заголовки Content-Type / Accept, коды ошибок;
- **фильтрация и пагинация** больших коллекций (поля, pageSize / pageToken, сортировка);
- **Partial Content:** HEAD → длина, GET с Range (докачка);
- **HATEOAS** — ссылки на доступные операции в ответе (но никто не поддерживает, формат не стандартизован);
- **версионирование** — старые клиенты работают; все версии параллельно (/v3/ в URL — частый, но не лучший способ).

[+] контракт — **OpenAPI** (бывший Swagger; генерация клиентов / заглушек); для коллекций — cursor-пагинация; где REST тесен — GraphQL (клиент выбирает поля) или gRPC (внутренние вызовы).

5. Принципы дизайна: самовосстановление

Сеть ненадёжна $\Rightarrow$ проектировать с учётом отказов. Виды (разные стратегии): *временные* (потеря пакетов), *постоянные* (сервис мёртв), *перегрузка*.

- **повтор** + **экспоненциальный откат** (но отличать временный отказ от постоянного — неверный пароль повтором не лечится; потому редко в библиотеках);
- **методы самодиагностики** (ping $\rightarrow$ 200 / 204; бизнес-статус) — для приложения и оркестратора;
- **изоляция ресурсов** (bulkhead) — лимиты от оркестратора;
- **буферизация очереди** при неравномерной нагрузке (длину ограничивать);
- **failover** на резерв (мин. 2 экземпляра; обратно — вручную; резерв в другом регионе);
- **checkpoints** долгих операций; **graceful degradation** (упал сервис картинок — заглушки);
- **намеренный ввод отказов** (раз в месяц грохать прод; **Chaos Monkey**).

[+] chaos engineering повзрослел: Gremlin, Chaos Mesh, LitmusChaos, AWS FIS (малый blast radius + гипотеза + наблюдаемость); в K8s самодиагностика — liveness / readiness / startup-пробы; повтор с jitter; библиотеки Polly (.NET), resilience4j.

6. Паттерн «Circuit Breaker»

При отказе из-за перегрузки лавина повторов от всех клиентов *усугубляет* ситуацию. **Circuit Breaker** (предохранитель) — прокси-объект с тремя состояниями:

- **Closed** (цепь замкнута) — запросы идут сервису; ошибка увеличивает счётчик; превысил порог $\rightarrow$ *Open*;
- **Open** (разомкнута) — запросы *не* идут, предохранитель сам бросает исключение; запускает таймер; по истечении $\rightarrow$ *Half-Open*;
- **Half-Open** (наполовину) — пропускает *немного* запросов: успех $\rightarrow$ *Closed* (счётчик сброшен), ошибка $\rightarrow$ *Open* (таймер перезапущен).

Сочетается с повторами, но полезно дать коду повторов узнать, что цепь разомкнута — повторять бессмысленно.

[+] готовые библиотеки Polly (.NET), resilience4j (Java) — комбинируют с retry, timeout, bulkhead в «политику устойчивости»; в микросервисах размыкание делают на уровне инфраструктуры — service mesh (Istio / Envoy).

7. Принципы дизайна: избыточность

Лучший способ отказоустойчивости — **избыточность** (несколько копий). Стоит денег $\Rightarrow$ сначала *бизнес-решение* — целевые показатели: **RTO** (макс. время от обнаружения отказа до восстановления), **RPO** (данные за какой период можно потерять — «как часто жать Ctrl-S»), **MTO** (сколько процесс может быть недоступен; успех — даже ручное исполнение).

Механизмы: **балансировщики** (stateless-сервис в 2–10 копий; упавшему не шлёт; + скорость); **репликация БД** (частота по RPO; обязательна); **регионы** (гео-распределение + DNS на ближайший; дорого); **шардирование** (размазать данные по БД). Аналогия RAID: репликация = **RAID 1** (копия для надёжности), шардирование = **RAID 0** (распределение для скорости; параллельные операции; потеря шарда = потеря *части*). Политика: по регионам / первой букве / хешу. Шардирование + репликация — вместе.

[+] облака дают **multi-AZ** почти даром, multi-region дорого; managed-БД реплицируются сами; **NewSQL** (Spanner, CockroachDB) — шардирование + репликация + почти-ACID под капотом.

8. Принципы дизайна: минимизация координации

Слишком тесная координация бьёт по производительности и надёжности. Приёмы:

- **событийный стиль** вместо прямых вызовов — **доменные события** (domain events), подписка и асинхронная обработка; источник не знает подписчиков (библиотека MediatR);
- **асинхронность** мультисервисных операций — иначе *каскад задержек*;
- **идемпотентные операции** — безопасно повторяются (запросы, «включить» / «выключить» — да; «переключить» — нет); при сбое другой обработчик берёт очередь и продолжает;
- **шардирование** — запросы не толкаются вокруг одной БД;
- **избегать распределённых транзакций** («никто не умеет») — выполнять локально, при ошибке — **компенсационная транзакция**; семантика **eventual consistency**.

[+] компенсации + eventual consistency сегодня — **паттерн Saga** (Гарсиа-Молина, Салем, 1987): цепочка локальных транзакций, при сбое компенсации в обратном порядке. Формы: **хореография** (на событиях, Kafka) и **оркестрация** (движок Temporal). Альтернатива блокирующему **2PC**; надёжная публикация — паттерн Outbox, ключи идиоматичности.

9. Command and Query Responsibility Segregation (CQRS)

CQS (Command and Query Separation) — команда *меняет* состояние и *ничего не возвращает*, запрос *возвращает* и *ничего не меняет* (хорошая практика ООП). **CQRS** — эволюция: для команд и запросов можно держать *разные*

схемы и хранилища (реляционная для чтения, другая для записи), что кратно ускоряет (у запросов нет конфликтов транзакций — они ничего не меняют). Предполагает явные **доменные команды** с бизнес-логикой.

Поддержание двух баз — через **eventual consistency**. Сильно помогает **немутабельность** данных (нет конфликтов записи, легко параллелить — как Kafka). Это подход **Event Sourcing** — хранить не состояние, а **набор событий**, приведших к нему; текущее состояние строится из начального + события; для скорости — read-only снапшоты, обновляемые из потока. Эффективно, когда событий и обработчиков много.

[+] CQRS применяют *точечно* (контексты с разным профилем чтения / записи); read-store — это **проекция** (read model) из событий. Event Sourcing + CQRS — фреймворки Axon (Java), EventStoreDB, часто с Kafka как журналом.

10. CAP-теорема, модели ACID и BASE

CAP-теорема: в распределённой системе не более двух из трёх — **Consistency** (во всех узлах данные согласованы), **Availability** (любой запрос завершается корректно, но без гарантии одинаковости ответов), **Partition tolerance** (потеря связи между узлами не портит корректность). **Р обязателен** (сеть ненадёжна) ⇒ выбор между **С** и **А** (традиционные СУБД на одной машине не устойчивы к разделению, зато дают С + А). Согласованность — остановить узлы до договорённости (ср. *задача византийских генералов*) — бьёт по доступности; голая доступность тоже плоха.

ACID (централизованные): **Atomicity** (не применится частично), **Consistency** (не рушит целостность), **Isolation** (параллельные не мешают), **Durability** (данные не теряются). **BASE** (распределённые): **Basically Available** (отказ узла → некорректный ответ его клиентам), **Soft-state** (состояние меняется само, согласованность не гарантируется), **Eventually consistent** (целостность лишь *когда-то*). ACID пытались перенести в распределёнку — CAP не дал; берут **BASE; eventual consistency** — часть BASE.

[+] CAP — только про поведение *при разделении*; точнее **PACELC** (Абади): *если Partition — А или С, иначе — Latency или Consistency*. NewSQL (Spanner, CockroachDB) и настраиваемая согласованность (Cassandra, DynamoDB) дают выбор точки компромисса.

11. Принципы дизайна: проектирование для обслуживания

Распределённые приложения трудно контролировать ⇒ сразу проектировать так, чтобы ими можно было *управлять и обслуживать* в проде — **Design for operations** (основа DevOps). Главный принцип — **делать всё наблюдаемым**:

- **логирование** всего + централизованный сбор (**Elastic Stack**: Logstash + Elasticsearch + Kibana; в сервисах — log4j / slf4j, log4net);
- **трассировка** — путь *одного запроса* через сервисы (нужна корреляция); лог — *горизонтальный* срез (один сервис), трасса — *вертикальный* (один запрос);
- **метрики и мониторинг** — запросов/с, ошибок/с, uptime, доля 5xx; выбор метрик — искусство, но мониторинг нужен;
- **стандартный формат** логов и метрик для всех сервисов;
- **автоматизировать всё** (развёртывание, мониторинг) — одной командой;
- **конфигурация как код** — хранить, версионировать, применять в CI/CD (Docker Compose, Kubernetes: YAML / JSON; docker compose up, kubectl apply); руками в работающую конфигурацию — плохо.

[+] стандарт наблюдаемости — **OpenTelemetry** (CNCF; единые SDK для *трёх столбов* — логи / метрики / трассы); стек Prometheus + Grafana + Loki + Tempo / Jaeger (сменил Elastic Stack); цели — SLI / SLO / SLA + error budget (SRE); конфигурация как код выросла в GitOps (ArgoCD, Flux) и IaC (Terraform).

Лекция 14. Развёртывание распределённых приложений

Экзаменационные вопросы — краткие ответы

1. Docker, Docker Compose

Docker — легковесная «ВМ», виртуализирующая *системные вызовы* (использует ядро ОС хоста): хуже изоляция, чем у обычных ВМ, но гораздо быстрее. Контейнер — виртуализированный *процесс* (внутри один процесс), не машина.

Образ (Image) — запускаемый образ приложения с окружением (как навороченный .exe); состоит из *read-only слоёв* (Alpine → рантайм → сервис → конфиг), слои *разделяются* между образами (экономия места / трафика).

Контейнер — запущенный экземпляр + *писчий* слой (гибнет при перезапуске). **Docker Hub** — репозиторий образов (как GitHub).

CLI: docker run (-d фон, -p host:container порт, -it интерактивно; нет образа — pull); ps; stop (SIGTERM → SIGKILL); exec (доп. процесс). **Dockerfile** — конфиг сборки: FROM, WORKDIR, COPY, RUN, EXPOSE, ENV, CMD/ENTRYPOINT. **Двухфазная сборка** (multi-stage): собрать в образе с SDK, скопировать *только результат* в тонкий рантайм-образ (FROM сбрасывает, AS запоминает слой).

Docker Compose — оркестратор из коробки (docker-compose.yml): services (образ, deploy: реплики, лимиты, restart_policy), ports, networks (реплики за балансировщиком). **Docker Swarm** — кластер из машин: swarm init (менеджер) → token → swarm join (узлы); на каждой машине балансировщик.

[+] стандарт **OCI** (эталон runc); Docker не единственный — Podman (без демона, rootless), containerd, Buildah/kaniko; Docker Desktop платен для компаний от 250 сотрудников (2021); Compose v2 — docker compose; Swarm вытеснен Kubernetes.

2. Kubernetes

Kubernetes — продвинутый оркестратор больших высоконагруженных систем; задачи те же, что у Compose + Swarm, но в разы функциональнее и сложнее (стандарт де-факто). Из Google Borg, open source, на Go.

Master (реплицирован): **etcd** (БД конфигурации), **Controller Manager** (связь с облаком, балансировщики), **Scheduler** (раскидывает контейнеры). **Node:** **kubelet** (запускает контейнеры, следит), **kube-proxy** (сеть), **cAdvisor** (метрики), **Pod** (набор Docker-контейнеров вместе — единица развёртывания).

Объекты (в etcd, применяются YAML): **Deployment** (деплой Pod-a; репликация, выкатка; создаёт ReplicaSet) → **ReplicaSet** → **Pod**; **Service** (прокси + балансировщик, фиксированный IP), **Ingress** (маршрутизация + TLS). kubectl apply — обновляет желаемое состояние, считает разницу. **Метки** — для выборки ресурсов.

Пробы: **livenessProbe** (жив ли — неудача ⇒ перезапуск), **readinessProbe** (готов ли — неудача ⇒ не слать запросы). **Стратегии обновления:** **Blue/Green** (новая версия рядом, переключить трафик; откат), **Rainbow** (несколько версий сразу), **Canary** (малый % трафика растёт до 100%; Istio). *Не используйте meз latest*. **Инструменты:** Helm (чарты), Dashboard, Prometheus, Clair (сканер), Velero (бэкап etcd). **Метрики:** RED (внешнее) и USE (внутреннее). Конфигурация развёртывания — *это код* (только декларативно, в систему контроля версий).

[+] в K8s 1.24 (2022) убрали dockershim — рантайм по умолчанию **containerd** (~95 %) или CRI-O (образы Docker работают по OCI); кластеры берут managed (EKS / GKE / AKS); GitOps (ArgoCD, Flux), Kustomize; безопасность — Trivy, Falco.

3. Облачная инфраструктура, AWS, Terraform

Содержать свои ресурсы обычно *накладнее* приложения ⇒ арендовать облако. Три категории: **IaaS** (Infrastructure — ВМ, диск, трафик; дальше сами), **PaaS** (Platform — готовые сервисы, напр. K8s-кластер; дешевле и быстрее, если знаете, что делаете), **SaaS** (Software — готовое приложение: Google Docs, облачные IDE). Модели свободно смешивают.

Провайдеры: **AWS** (лидер), **Azure** (интеграция с Microsoft), **Google Cloud** (GKE — старейший K8s; Colab), **Yandex.Cloud** (персональные данные в РФ по закону), **Heroku** (для новичков). **Экосистема AWS:** вычисления — EC2 (ВМ), ECS (контейнеры) + ECR; сеть — VPC, ELB, API Gateway; хранение — EFS, EBS; БД — RDS, DynamoDB (NoSQL), OpenSearch.

Infrastructure as Code (IaC): манифесты K8s описывают развёртывание *на готовой* инфраструктуре; IaC описывает кодом *саму инфраструктуру* («нужен K8s-кластер из 5 узлов, балансировщик, NoSQL») — независимо от провайдера, в систему контроля версий, применить к выбранному провайдеру. Даёт *воспроизводимость* и смену провайдера (в идеале — одной командой). **Terraform** — инструмент IaC (~135 провайдеров, язык HCL); специфика провайдера всё же нужна.

[+] доли рынка сместились: AWS уже не ~50 %, а около **30 %** (Q2 2025), Azure ~20 %, GCP ~13 % (отрыв AWS сокращается). Terraform в 2023 г. сменил лицензию (BSL) — форк **OpenTofu** (Linux Foundation); альтернатива **Pulumi** (IaC на обычных языках) и AWS CDK; Heroku закрыл free-tier (2022); serverless (Lambda), FinOps.

Лекция 15. Основы сетевой безопасности

Экзаменационные вопросы — краткие ответы

1. Основные понятия. Шифрование с симметричным ключом

Аутентификация (личность — тот ли пользователь; часто взаимна) $\neq$ **авторизация** (права на операцию). **Шифрование** — конфиденциальность; также важны **целостность** (не изменено) и **актуальность** (не replay). Взлом «в лоб» безнадежен — ломают *социально* и *изнутри*; цель — взлом *дороже* *выгоды*. **Главное правило** — **свои шифры / протоколы не писать**.

Модель: $C = E_K(P)$ (P — plaintext, C — ciphertext, K — ключ); дешифр. D . **Принцип Керкгоффса:** алгоритм известен, секретен только ключ. Стойкость — *длиной ключа*.

Симметричный шифр — один ключ, очень быстрый $\Rightarrow$ шифрует *весь* трафик; использовать как чёрный ящик (OpenSSL, не писать своё). **DES** (1975, 56 бит, сломан) $\rightarrow$ **3DES** ($E_{K3}(D_{K2}(E_{K1}(P)))$) $\rightarrow$ **AES** (2001, 128/192/256, открытый конкурс, основной).

Режимы: **one-time pad** (хот со случайным ключом длины текста — идеален, но ключ не передать). **ECB** (блоки независимо — уязвим к статистике и перестановке блоков; *не применяется*); **CBC** (хот с предыдущим шифрблоком; первый — с **IV**; порча бита рушит всё после); **SCM** (поточковый keystream, хот; нельзя повторять keystream: $(P \oplus K) \oplus (Q \oplus K) = P \oplus Q$); **CTR** (шифруем $IV, IV+1, \dots$ — произвольный доступ).

[+] 3DES запрещён NIST после 2023; стандарт — AES в **AEAD**-режимах (AES-GCM, ChaCha20-Poly1305: шифрование + целостность сразу).

2. Алгоритм Диффи-Хеллмана. Шифрование с открытым ключом

Диффи-Хеллман — общий секретный ключ *по открытому каналу*. Алиса и Боб выбирают случайные x, y и большие n, g ; шлют $g^x \bmod n$ и $g^y \bmod n$; общий ключ $g^{xy} \bmod n$ (оба вычисляют, злоумышленник — нет: *дискретное логарифмирование* полиномиально не решается). *Не защищает от MITM*: без аутентификации злоумышленник делает ДН дважды (своё z) и получает два ключа — ДН даёт ключ, но не аутентичность. Где: приватные чаты Telegram, HTTPS.

Шифрование с открытым ключом (асимметричное): разные ключи; $D(E(P)) = P$, ключ от D нельзя получить из ключа от E . E — *открытый* (публикуется), D — *закрытый* (секрет). Боб шифрует открытым ключом Алисы E_A , Алиса дешифрует своим D_A — секретами не обмениваются. Схемы: **RSA** (множители), **ElGamal** (дискретные логарифмы), **эллиптические**. Медленные $\Rightarrow$ ими аутентифицируются и выбирают симметричный ключ.

[+] классический ДН вытеснен **ECDH / X25519**; в TLS 1.3 эфемерный ECDHE (forward secrecy). Угроза — *квантовый компьютер* (алгоритм Шора ломает RSA / ECC); NIST в 2024 стандартизовал пост-квантовый **ML-KEM** (бывш. Kyber); браузеры катят гибрид X25519 + ML-KEM.

3. Цифровые подписи, сертификаты

Цифровая подпись — гарантия *авторства* и *неизменности* (не конфиденциальности). Алиса шифрует своим *закрытым* ключом D_A ; Боб применяет открытый E_A — если осмысленно, автор Алиса (и не отопрётся). Шифровать всё дорого $\Rightarrow$ подписывают **дайджест** — криптографический *хеш* (быстрый, однозначный, малое изменение меняет до неузнаваемости, необратим). Адресату — открытый текст + подписанный хеш; адресат считает хеш сам и сверяет. **Хеши:** MD5 (сломан), SHA (SHA-1 сломан; практически SHA-2, SHA-3). **HMAC** — хеш от данных + секретного ключа.

Сертификат — сообщение «предъявитель владеет доменом», подписанное **CA** (Certificate Authority), которому доверяют обе стороны (формат **X.509**). CA верхнего уровня подписывает CA ниже — *цепочка доверия* до **корневого** (root) сертификата, вшитого в ОС / браузер (передают всю цепочку). Срок ограничен, CA может отозвать.

Самоподписанный — сам сгенерировал (Fiddler); **Let's Encrypt** — бесплатное CA для домена. Применяются: HTTPS, подписывание кода (SmartScreen, Apple), .NET-сборки («сильное имя»).

[+] SHA-1 полностью сломан (SHATTERED 2017) $\rightarrow$ SHA-256 / SHA-3, MD5 — никогда; выдача автоматизирована **ACME** (Let's Encrypt), сроки жизни резко сокращают; на горизонте пост-квантовые подписи **ML-DSA**.

4. Аутентификация. OAuth 2

Challenge-Response (общий секретный ключ): Боб шлёт **challenge** — **nonce** (случайное, одноразовое); Алиса возвращает зашифрованным; Боб сверяет; затем симметрично. *Зеркальная атака*: в 3-сообщенческой версии злоумышленник открывает вторую сессию и заставляет Боба зашифровать его же challenge — внешне корректный протокол уязвим $\Rightarrow$ *свои протоколы не писать* (лечится HMAC).

Basic Authentication: по HTTPS (сервер уже доказал идентичность) клиент шлёт логин / пароль открытым текстом. Сервер выдаёт **Access Token** (живёт ограниченно). Пароли *не хранят* — хранят **хеш**; против «радужных таблиц» добавляют **соль** (salt). **Аутентификация открытым ключом:** сервер хранит *открытые* ключи клиентов (SSH на GitHub).

OAuth 2 (авторизация) — доступ к ресурсу *без раскрытия пароля* владельцу ресурса (вход по Google / VK). Роли: **Client**, **Resource Owner**, **Authorization Server**, **Resource Server**. Режим **Authorization Code**: Client *перенаправляет* владельца на AS (логин / пароль видит только AS); AS выдаёт **Access Token** (со **Scope**; протухает быстро) и **Refresh Token** (дольше живёт, продлевается / отзывается). Пример — Google OAuth 2.0.

[+] пароли — **Argon2id** / bcrypt (memory-hard), не голый SHA; **OAuth 2.1** (PKCE обязателен, implicit удалён);

OpenID Connect (аутентификация поверх OAuth); **passkeys** (WebAuthn / FIDO2).